

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC TÂY ĐÔ
KHOA KỸ THUẬT - CÔNG NGHỆ**

Trí tuệ - Sáng tạo - Năng động - Đổi mới

KHÓA LUẬN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG RAG NHẬN BIẾT CẤU TRÚC
VÀ THỜI GIAN HIỆU LỰC ĐỂ HỖ TRỢ TRA CỨU
PHÁP LUẬT GIAO THÔNG VIỆT NAM VỚI TRÍCH
DẪN CÓ THỂ KIỂM CHỨNG**

Chuyên ngành: Công nghệ thông tin

Ngành: Công nghệ thông tin

Sinh viên thực hiện: Quách Trường Phúc

Mã số sinh viên: [Mã số sinh viên]

Người hướng dẫn: [Tên giảng viên hướng dẫn]

Cần Thơ, tháng 9 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC TÂY ĐÔ
KHOA KỸ THUẬT - CÔNG NGHỆ

KHÓA LUẬN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG RAG NHẬN BIẾT CẤU TRÚC
VÀ THỜI GIAN HIỆU LỰC ĐỂ HỖ TRỢ TRA CỨU
PHÁP LUẬT GIAO THÔNG VIỆT NAM VỚI TRÍCH
DẪN CÓ THỂ KIỂM CHỨNG**

*A Structure-Aware and Temporal RAG System for Vietnamese Traffic Law
Question Answering with Verifiable Citations*

Chuyên ngành: Công nghệ thông tin
Ngành: Công nghệ thông tin
Sinh viên thực hiện: Quách Trương Phúc
Mã số sinh viên: [Mã số sinh viên]
Người hướng dẫn: [Tên giảng viên hướng dẫn]

Cần Thơ, tháng 9 năm 2026

LỜI CẢM ƠN

Tác giả xin gửi lời cảm ơn chân thành đến [Tên giảng viên hướng dẫn] đã dành thời gian hướng dẫn, góp ý và hỗ trợ trong quá trình xác định bài toán, lựa chọn phương pháp và xây dựng hệ thống. Các nhận xét chuyên môn đã giúp đề tài được điều chỉnh từ một chatbot hỏi đáp tài liệu tổng quát thành một nghiên cứu có phạm vi rõ ràng về truy xuất pháp luật giao thông theo cấu trúc, quan hệ tham chiếu và thời gian hiệu lực.

Tác giả trân trọng cảm ơn quý thầy cô Khoa Kỹ thuật - Công nghệ, Trường Đại học Tây Đô đã cung cấp nền tảng kiến thức và tạo điều kiện để khóa luận được thực hiện. Tác giả cũng cảm ơn gia đình, bạn bè và những người đã hỗ trợ kiểm tra tài liệu, đối chiếu nội dung văn bản pháp luật, trao đổi tình huống sử dụng và động viên trong thời gian triển khai đề tài.

Khóa luận còn có thể tồn tại thiếu sót do giới hạn về thời gian, quy mô corpus và kinh nghiệm nghiên cứu. Tác giả mong nhận được ý kiến góp ý để hệ thống và phương pháp đánh giá tiếp tục được hoàn thiện.

LỜI CAM ĐOAN

Tôi xin cam đoan rằng khóa luận này là do chính tôi thực hiện, không sao chép dưới bất kỳ hình thức nào hay thuê hoặc nhờ người khác thực hiện.

Dữ liệu và kết quả phân tích trong khóa luận bảo đảm tính chính xác, khách quan và trung thực, không có bất kỳ sự ngụy tạo hoặc điều chỉnh kết quả nghiên cứu theo chủ quan của tác giả. Những nội dung, phương pháp và tài liệu được kế thừa từ công trình khác đều được trích dẫn trong danh mục tài liệu tham khảo.

Cần Thơ, ngày ... tháng ... năm 2026

Tác giả

(Ký tên và ghi rõ họ và tên)

Quách Trương Phúc

TÓM TẮT

Khóa luận nghiên cứu và xây dựng một hệ thống Retrieval-Augmented Generation (RAG) hỗ trợ tra cứu pháp luật giao thông đường bộ Việt Nam. Khác với mô hình hỏi đáp tài liệu thông thường, hệ thống được thiết kế để nhận biết cấu trúc Chương, Mục, Điều, Khoản và Điểm kèm nhãn Điểm tiếng Việt (a) b) c) d) đ) e)); mô hình hóa quan hệ tham chiếu chéo giữa các quy định; quản lý khoảng thời gian hiệu lực của từng provision; đồng thời kiểm chứng trích dẫn theo nhiều tầng trước khi trả kết quả.

Pipeline ingestion sử dụng Parser Router định tuyến tài liệu qua Docling hoặc MinerU theo đặc tính tài liệu và quality gate, sau đó chuẩn hóa sang Canonical Document IR do dự án sở hữu. Legal Structure Extractor trích xuất phân cấp pháp luật Việt Nam và tạo định danh provision ổn định; Legal Reference Resolver và Temporal and Amendment Resolver xác định quan hệ tham chiếu và khoảng hiệu lực. PostgreSQL là nguồn chân lý dữ liệu pháp lý; Qdrant là index dẫn xuất lưu dense vector, sparse BM25 và payload phục vụ truy xuất lai. Ingestion chạy nền qua Redis và Dramatiq; PDF nguồn và artifact được lưu trong MinIO.

Pipeline hỏi đáp sử dụng LangGraph như một workflow có đường đi xác định trước, gồm query understanding, temporal resolution, query expansion, retrieval đa kênh (exact lookup, dense, sparse) với RRF fusion, reranking, mở rộng ngữ cảnh pháp lý theo quan hệ và Evidence Completeness Gate. Gemini 3.5 Flash sinh câu trả lời có cấu trúc cấp claim, trong đó mỗi claim chỉ được tham chiếu provision_id thuộc context. Bộ kiểm chứng sáu tầng (schema, citation ID, temporal, numeric grounding, claim support, evidence completeness) quyết định trả verified answer hay abstention với lý do chuẩn. Khi bằng chứng không đủ, hệ thống từ chối trả lời thay vì tìm kiếm web tự do.

Nghiên cứu đề xuất gold set 200 câu đã review, chia 40 development, 40 validation và 120 final test, cùng bốn suite thí nghiệm A-D: parser benchmark (P1-P3), embedding benchmark (E1-E3), retrieval ablation (R1-R10) và generation/verification ablation (G1-G7). Các metric chính gồm Recall@k, MRR@10, nDCG@10, Citation Precision/Recall/F1, Temporal Validity Accuracy, Numeric Grounding Accuracy, Evidence Completeness và Abstention F1. Langfuse phục vụ observability ngoài đường tới hạn; RAGFlow chỉ được dùng làm baseline so sánh bên ngoài.

BẢN THẢO KỸ THUẬT - CHƯA CÓ KẾT QUẢ THỰC NGHIỆM CUỐI.

Các bảng kết quả trong Chương 6 phải được thay bằng số liệu từ evaluation run đã đóng băng trước khi nộp.

Từ khóa: Retrieval-Augmented Generation, pháp luật giao thông Việt Nam, truy xuất theo thời gian, hybrid retrieval, kiểm chứng trích dẫn, evidence completeness, Parser Router, Canonical Document IR.

ABSTRACT

This thesis investigates and develops a Retrieval-Augmented Generation system for Vietnamese road traffic law question answering. Unlike a conventional document chatbot, the proposed system is designed to preserve the legal hierarchy of chapters, sections, articles, clauses, and points including Vietnamese point labels (a) b) c) d) đ) e)); to model cross-references between provisions; to represent the effective period of each provision; and to verify citations through multiple layers before returning an answer.

The offline ingestion pipeline uses a Parser Router that dispatches documents to Docling or MinerU based on document characteristics and quality gates, then normalizes the output into a project-owned Canonical Document IR. A dedicated Legal Structure Extractor recognizes the Vietnamese legal hierarchy and generates stable provision identifiers; a Legal Reference Resolver and a Temporal and Amendment Resolver derive cross-reference relations and effective intervals. PostgreSQL is the legal source of truth, while Qdrant is a rebuildable retrieval index holding dense vectors, sparse BM25, and payload metadata. Ingestion runs in the background with Redis and Dramatiq; source PDFs and artifacts are stored in MinIO.

The online query pipeline is a controlled LangGraph workflow with predefined branches: query understanding, temporal resolution, query expansion, multi-channel retrieval (exact lookup, dense, sparse) fused by Reciprocal Rank Fusion, reranking, relation-aware legal context expansion, and an Evidence Completeness Gate. Gemini 3.5 Flash generates a claim-level structured answer whose claims may only reference provision identifiers from the retrieved context. A six-layer verifier (schema, citation ID, temporal, numeric grounding, claim support, evidence completeness) decides between a verified answer and abstention with a standard reason code. The system abstains when evidence is insufficient instead of relying on unrestricted web search.

The study proposes a reviewed gold set of 200 questions split into 40 development, 40 validation, and 120 final test cases, together with four experiment suites: parser benchmark (P1-P3), embedding benchmark (E1-E3), retrieval ablation (R1-R10), and generation/verification ablation (G1-G7). Primary metrics include Recall@k, MRR@10, nDCG@10, Citation Precision/Recall/F1, Temporal Validity Accuracy, Numeric Grounding Accuracy, Evidence Completeness, and Abstention F1. Langfuse provides observability outside the correctness-critical path; RAGFlow is used only as an external comparison baseline.

BẢN THẢO KỸ THUẬT - CHƯA CÓ KẾT QUẢ THỰC NGHIỆM CUỐI.

*Các bảng kết quả trong Chương 6 phải được thay bằng số liệu từ
evaluation run đã đóng băng trước khi nộp.*

Keywords: *Retrieval-Augmented Generation, Vietnamese traffic law, temporal retrieval, hybrid retrieval, citation verification, evidence completeness, Parser Router, Canonical Document IR.*

Mục lục

<i>Lời cảm ơn</i>	<i>i</i>
<i>Lời cam đoan</i>	<i>ii</i>
<i>Tóm tắt</i>	<i>iii</i>
<i>Abstract</i>	<i>v</i>
<i>Danh mục bảng</i>	<i>xi</i>
<i>Danh mục hình</i>	<i>xii</i>
<i>Danh mục ký hiệu và từ viết tắt</i>	<i>xiii</i>
<i>Nhận xét của giảng viên hướng dẫn</i>	<i>xv</i>
1 Tổng Quan Đề Tài	1
1.1 Bối cảnh	1
1.2 Vấn đề nghiên cứu	1
1.3 Mục tiêu	2
1.3.1 Mục tiêu tổng quát	2
1.3.2 Mục tiêu cụ thể	2
1.4 Câu hỏi nghiên cứu	3
1.5 Đối tượng và phạm vi	4
1.6 Phương pháp thực hiện	4
1.7 Đóng góp dự kiến	5
1.8 Bố cục khóa luận	5
2 Cơ Sở Lý Thuyết và Nghiên Cứu Liên Quan	6
2.1 Mô hình ngôn ngữ lớn	6
2.2 Retrieval-Augmented Generation	6
2.3 Dense retrieval và embedding	6
2.4 Sparse retrieval và BM25	7
2.5 Hybrid retrieval và RRF	7

2.6	<i>Chunking tài liệu</i>	7
2.7	<i>Document parsing và provenance</i>	8
2.8	<i>Temporal data và phiên bản pháp luật</i>	8
2.9	<i>Citation verification và abstention</i>	9
2.10	<i>Evidence completeness</i>	9
2.11	<i>Đánh giá hệ thống RAG</i>	10
2.12	<i>Khoảng trống nghiên cứu</i>	10
3	<i>Phân Tích Yêu Cầu và Phương Pháp Nghiên Cứu</i>	11
3.1	<i>Tác nhân và use case</i>	11
3.2	<i>Yêu cầu chức năng</i>	11
3.3	<i>Yêu cầu phi chức năng</i>	13
3.4	<i>Thiết kế corpus</i>	14
3.5	<i>Thiết kế gold set</i>	14
3.6	<i>Thiết kế thí nghiệm</i>	15
3.7	<i>Metric</i>	16
3.8	<i>Chống leakage và bảo đảm tái lập</i>	17
3.9	<i>Đe dọa đến tính hợp lệ</i>	17
4	<i>Thiết Kế Hệ Thống</i>	19
4.1	<i>Nguyên tắc thiết kế</i>	19
4.2	<i>Kiến trúc tổng quan</i>	19
4.3	<i>Pipeline ingestion ngoại tuyến</i>	20
4.4	<i>Parser Router và Canonical Document IR</i>	21
4.5	<i>Legal Structure Extractor</i>	21
4.6	<i>Mô hình dữ liệu</i>	22
4.6.1	<i>LegalDocument và DocumentVersion</i>	22
4.6.2	<i>LegalProvision</i>	22
4.6.3	<i>ProvisionReference và DocumentRelation</i>	23
4.6.4	<i>LegalEffectEvent</i>	23
4.7	<i>PostgreSQL schema</i>	23
4.8	<i>Qdrant collection</i>	24
4.9	<i>Retrieval đa tầng</i>	25
4.9.1	<i>Query Understanding và temporal resolution</i>	25
4.9.2	<i>Query Expansion</i>	25
4.9.3	<i>Ba kênh retrieval, RRF và reranking</i>	25
4.9.4	<i>Legal context expansion</i>	26
4.10	<i>Evidence Completeness Gate</i>	26
4.11	<i>Workflow hỏi đáp</i>	26
4.12	<i>Structured generation</i>	27

4.13	<i>Verification sáu tầng</i>	27
4.14	<i>Abstention</i>	28
4.15	<i>Langfuse observability</i>	29
4.16	<i>Background ingestion và object storage</i>	29
4.17	<i>API và giao diện</i>	29
4.18	<i>Bảo mật</i>	30
5	<i>Cài Đặt và Triển Khai</i>	31
5.1	<i>Tech stack</i>	31
5.2	<i>Cấu trúc source code</i>	31
5.3	<i>Cài đặt ingestion</i>	32
5.4	<i>Cài đặt retrieval</i>	32
5.5	<i>Cài đặt verifier</i>	33
5.6	<i>Giao diện người dùng</i>	33
5.7	<i>Docker Compose</i>	34
5.8	<i>Migration, backup và restore</i>	35
5.9	<i>CI/CD và quality gate</i>	35
5.10	<i>RAGFlow baseline (external)</i>	35
5.11	<i>Ràng buộc phần cứng</i>	36
5.12	<i>Kế hoạch thời gian</i>	36
6	<i>Kiểm Thử và Đánh Giá</i>	37
6.1	<i>Chiến lược kiểm thử</i>	37
6.2	<i>Test extraction và parser</i>	37
6.3	<i>Test temporal và citation</i>	37
6.4	<i>Thiết lập evaluation</i>	38
6.5	<i>Thiết kế thí nghiệm các suite</i>	38
6.5.1	<i>Suite A - Parser benchmark (P1-P3)</i>	38
6.5.2	<i>Suite B - Embedding benchmark (E1-E3)</i>	38
6.5.3	<i>Suite C - Retrieval ablation (R1-R10)</i>	38
6.5.4	<i>Suite D - Generation và verification ablation (G1-G7)</i>	38
6.6	<i>Kết quả extraction</i>	38
6.7	<i>Kết quả retrieval</i>	39
6.8	<i>Kết quả temporal và citation</i>	40
6.9	<i>Kết quả generation và abstention</i>	40
6.10	<i>Latency, tài nguyên và chi phí</i>	41
6.11	<i>Phân tích lỗi</i>	41
6.12	<i>Diễn giải kết quả</i>	41
6.13	<i>Tiêu chí release</i>	41

7	Kết Luận và Hướng Phát Triển	42
7.1	Kết luận	42
7.2	Ưu điểm	42
7.3	Hạn chế	43
7.4	Hướng phát triển	43
7.5	Bài học phương pháp	43
	Tài Liệu Tham Khảo	44
A	Hướng Dẫn Chạy Demo	46
A.1	Khởi động hạ tầng	46
A.2	Kịch bản demo	46
B	Schema Gold Set	47
C	Checklist Trước Khi Nộp	48

Danh sách bảng

1.1	Câu hỏi nghiên cứu	3
1.2	Phân định phạm vi	4
2.1	So sánh định hướng nghiên cứu liên quan	10
3.1	Yêu cầu chức năng P0 (rút gọn)	11
3.2	Suite A - Parser benchmark	15
3.3	Suite B - Embedding benchmark	15
3.4	Suite C - Retrieval ablation	16
3.5	Suite D - Generation và verification ablation	16
3.6	Nhóm metric đánh giá	17
4.1	Các bảng dữ liệu chính	24
4.2	Sáu tầng verification	28
4.3	API chính	30
5.1	Công nghệ triển khai	31
5.2	Mốc triển khai chính	36
6.1	Kết quả hierarchy (Suite A) - chờ final evaluation	39
6.2	Kết quả các chỉ số bổ sung (Suite A) - chờ final evaluation	39
6.3	Kết quả retrieval - chờ final evaluation	40
6.4	Kết quả temporal và citation - chờ final evaluation	40
6.5	Kết quả generation - chờ final evaluation	40
6.6	Kết quả vận hành - chờ benchmark	41

Danh sách hình vẽ

<i>2.1</i>	<i>Cấu trúc phân cấp của một đơn vị pháp lý</i>	<i>8</i>
<i>3.1</i>	<i>Các tác nhân chính của hệ thống</i>	<i>11</i>
<i>4.1</i>	<i>Kiến trúc logic tổng thể của hệ thống</i>	<i>19</i>
<i>4.2</i>	<i>Pipeline xử lý và review tài liệu</i>	<i>20</i>
<i>4.3</i>	<i>Controlled workflow cho online query</i>	<i>26</i>
<i>5.1</i>	<i>Màn hình chat với bộ chọn ngày, loại phương tiện và citation panel</i>	<i>33</i>
<i>5.2</i>	<i>Màn hình so sánh quy định giữa hai thời điểm</i>	<i>34</i>
<i>5.3</i>	<i>Source drawer hiển thị passage, trang và hierarchy</i>	<i>34</i>
<i>5.4</i>	<i>Kiến trúc triển khai local với bany service</i>	<i>35</i>

DANH MỤC KÝ HIỆU VÀ TỪ VIẾT TẮT

<i>Ký hiệu</i>	<i>Diễn giải</i>
<i>AI</i>	<i>Artificial Intelligence - Trí tuệ nhân tạo</i>
<i>API</i>	<i>Application Programming Interface</i>
<i>BM25</i>	<i>Best Matching 25 - thuật toán xếp hạng truy xuất từ khóa</i>
<i>Canonical IR</i>	<i>Canonical Document IR - biểu diễn tài liệu trung gian parser-neutral của dự án</i>
<i>CI/CD</i>	<i>Continuous Integration / Continuous Delivery</i>
<i>CRUD</i>	<i>Create, Read, Update, Delete</i>
<i>Dramatiq</i>	<i>Thư viện background processing cho Python, chạy worker ingestion</i>
<i>E2E</i>	<i>End-to-End</i>
<i>Evidence Gate</i>	<i>Evidence Completeness Gate - cổng kiểm tra tính đầy đủ bằng chứng trước khi sinh câu trả lời</i>
<i>F1</i>	<i>F1-score - trung bình điều hòa của Precision và Recall</i>
<i>HITL</i>	<i>Human-in-the-Loop - con người trong vòng lặp (ở khâu review ingestion)</i>
<i>ID</i>	<i>Identifier - định danh</i>
<i>IR</i>	<i>Intermediate Representation - biểu diễn trung gian (Canonical Document IR)</i>
<i>JSON</i>	<i>JavaScript Object Notation</i>
<i>Langfuse</i>	<i>Nền tảng observability, quản lý prompt và experiment cho LLM</i>
<i>LegalProvision</i>	<i>Đơn vị pháp lý (Điều, Khoản, Điểm) kèm khoảng hiệu lực và provenance</i>
<i>LLM</i>	<i>Large Language Model - mô hình ngôn ngữ lớn</i>
<i>MinIO</i>	<i>Object storage S3-compatible</i>
<i>MRR</i>	<i>Mean Reciprocal Rank</i>
<i>nDCG</i>	<i>Normalized Discounted Cumulative Gain</i>
<i>OCR</i>	<i>Optical Character Recognition</i>
<i>P0/P1</i>	<i>Mức ưu tiên bắt buộc / mở rộng</i>

<i>Parser Router</i>	<i>Bộ định tuyến parser: Docling chính, MinerU phụ/fallback</i>
<i>PII</i>	<i>Personally Identifiable Information</i>
<i>RAG</i>	<i>Retrieval-Augmented Generation</i>
<i>RAGFlow</i>	<i>Công cụ RAG mã nguồn mở, chỉ dùng làm baseline so sánh bên ngoài</i>
<i>RRF</i>	<i>Reciprocal Rank Fusion</i>
<i>SDK</i>	<i>Software Development Kit</i>
<i>SHA</i>	<i>Secure Hash Algorithm</i>
<i>SQL</i>	<i>Structured Query Language</i>
<i>SSE</i>	<i>Server-Sent Events</i>
<i>VLM</i>	<i>Vision-Language Model</i>

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

CHƯƠNG 1

TỔNG QUAN ĐỀ TÀI

1.1 Bối cảnh

Văn bản pháp luật giao thông đường bộ có ảnh hưởng trực tiếp đến hoạt động hằng ngày của người dân. Tuy nhiên, việc tra cứu thường đòi hỏi người dùng xác định đúng loại văn bản, số hiệu, Điều, Khoản, Điểm và trạng thái hiệu lực. Một quy định về mức phạt có thể được ban hành trong một Nghị định, được sửa đổi bởi văn bản khác và được thay thế ở một thời điểm cụ thể. Vì vậy, câu trả lời đúng ở năm 2023 chưa chắc đúng ở năm 2026.

Cơ sở dữ liệu quốc gia về văn bản pháp luật cung cấp toàn văn, thuộc tính, lịch sử và văn bản liên quan. Ví dụ, Luật số 36/2024/QH15 có hiệu lực từ ngày 01/01/2025 [1]; Nghị định số 168/2024/NĐ-CP cũng có hiệu lực từ ngày 01/01/2025 và quy định xử phạt vi phạm hành chính, trừ điểm và phục hồi điểm giấy phép lái xe trong lĩnh vực giao thông đường bộ [2]. Trước thời điểm đó, nhiều tình huống xử phạt được tra cứu theo Nghị định số 100/2019/NĐ-CP và các văn bản sửa đổi [3]. Điều này làm cho thời gian hiệu lực và quan hệ giữa các văn bản trở thành dữ liệu bắt buộc thay vì metadata phụ.

Các mô hình ngôn ngữ lớn có khả năng diễn đạt và tổng hợp tốt, nhưng không bảo đảm tự động rằng số hiệu hoặc trích dẫn được sinh ra là có thật. LegalCiteBench cho thấy hướng dẫn bằng prompt về sự không chắc chắn có thể giảm một phần câu trả lời tự tin sai, nhưng không tự cải thiện độ đúng của citation khi thiếu grounding bên ngoài [4]. Do đó, một hệ thống pháp luật cần ràng buộc mô hình bằng dữ liệu truy xuất và một lớp kiểm chứng xác định.

1.2 Vấn đề nghiên cứu

Một pipeline RAG cơ bản thường gồm bốn bước: chuyển tài liệu thành text, chia thành chunk, truy xuất chunk gần với câu hỏi và yêu cầu LLM tạo câu trả lời. Pipeline này còn bẫy hạn chế khi áp dụng cho pháp luật:

1. **Mất cấu trúc pháp lý:** fixed-token chunking có thể tách phần dẫn của Khoản khỏi các Điểm bên dưới hoặc gộp nhiều điều khoản không liên quan. Ranh giới pháp lý phải trùng ranh giới trích dẫn; nhãn Điểm tiếng Việt (kể cả đ)) không thể được xử lý bằng giả định bảng chữ cái đơn giản.
2. **Không nhận biết thời gian:** vector search chỉ đo độ gần ngữ nghĩa, không biết văn bản đã hết hiệu lực hay chưa có hiệu lực tại ngày được hỏi.
3. **Bỏ sót theo định danh chính xác:** retrieval thuần vector có thể bỏ sót số hiệu

văn bản, số Điều, số Khoản hoặc cụm từ pháp lý có tính quyết định.

- 4. Quan hệ tham chiếu chéo chưa được mô hình hóa: một Điều dẫn chiếu Điều khác, quy định xử phạt thường gắn kèm quy định trừ điểm; hệ thống chỉ trả lời được nửa thông tin nếu không mở rộng ngữ cảnh theo quan hệ.*
- 5. Câu hỏi đa bằng chứng: một câu hỏi có thể yêu cầu vừa mức phạt vừa số điểm trừ; hệ thống không được âm thầm trả lời một nửa để của câu hỏi.*
- 6. Citation không kiểm chứng: mô hình có thể gõ số Điều hoặc số hiệu không tồn tại, hoặc ghép nội dung đúng với provision_id sai. Một citation tồn tại chưa đủ; số liệu trong claim còn phải khớp bằng chứng.*
- 7. Đánh giá không tách lớp: một điểm tổng hợp không chỉ ra lỗi đến từ parser, extraction, retrieval, temporal filter, generation hay verification.*

Vấn đề trung tâm của đề tài được phát biểu như sau:

Làm thế nào để xây dựng một hệ thống RAG truy xuất đúng đơn vị pháp lý, áp dụng đúng phiên bản văn bản tại thời điểm được hỏi, mở rộng đúng các quy định liên quan và chỉ trả về các nhận định có trích dẫn kiểm chứng được?

1.3 Mục tiêu

1.3.1 Mục tiêu tổng quát

Xây dựng và đánh giá một hệ thống RAG nhận biết cấu trúc, quan hệ tham chiếu và thời gian hiệu lực để hỗ trợ tra cứu pháp luật giao thông đường bộ Việt Nam bằng tiếng Việt.

1.3.2 Mục tiêu cụ thể

- 1. Xây dựng Parser Router định tuyến tài liệu qua Docling hoặc MinerU theo đặc tính tài liệu và quality gate.*
- 2. Thiết kế Canonical Document IR parser-neutral làm biểu diễn trung gian cho toàn bộ phân tích pháp lý.*
- 3. Xây dựng Legal Structure Extractor nhận diện phân cấp Chương, Mục, Điều, Khoản, Điểm với nhãn tiếng Việt a) b) c) d) đ) e) và giữ lại Điểm ngăn hợp lệ.*
- 4. Mô hình hóa quan hệ tham chiếu chéo và quan hệ hiệu lực bằng PostgreSQL, không dùng Neo4j.*
- 5. Biểu diễn văn bản và điều khoản bằng LegalDocument và LegalProvision, kèm định danh ổn định và khoảng hiệu lực [effective_from, effective_to).*

6. Kết hợp *exact lookup*, *dense retrieval* và *sparse BM25* trong Qdrant bằng *RRF*, kèm *reranking* và mở rộng ngữ cảnh pháp lý.
7. Xây dựng *evidence plan* và *Evidence Completeness Gate* để không trả lời nửa vời cho câu hỏi đa bằng chứng.
8. Sinh câu trả lời có cấu trúc cấp *claim* và kiểm chứng *citation* theo *provision_id* ở sáu tầng.
9. Thiết kế *abstention* có lý do chuẩn khi thiếu dữ liệu hoặc *verification* thất bại.
10. Xây dựng *gold set* 200 câu và chạy bốn suite thí nghiệm A-D.
11. Thiết lập *observability* qua *Langfuse* ngoài đường tới hạn và *baseline* so sánh *RAGFlow* bên ngoài.

1.4 Câu hỏi nghiên cứu

Bảng 1.1: Câu hỏi nghiên cứu

Mã	Câu hỏi
RQ1	<i>Docling, MinerU và Parser Router khác nhau thế nào về chất lượng trích xuất cấu trúc pháp luật Việt Nam (Article/Clause/Point P/R/F1, Short Point Recall, Vietnamese đ) Recall, Parent Context Completeness, Provenance Coverage)?</i>
RQ2	<i>Embedding nào trong ba ứng viên (Gemini Embedding 2, Jina Embeddings v5 text-nano, Jina Embeddings v5 text-small) cho kết quả retrieval tốt nhất trên câu hỏi pháp luật tiếng Việt (Recall@10, MRR@10, nDCG@10)?</i>
RQ3	<i>Từng tầng của retrieval đa tầng (exact lookup, sparse/RRF, query normalization, multi-query rewrite, conditional HyDE, reranker, parent/sibling expansion, cross-reference expansion, temporal filter) cải thiện Recall@k, MRR và nDCG ở mức nào?</i>
RQ4	<i>Temporal filtering làm giảm tỷ lệ trích dẫn sai phiên bản (Temporal Leakage Rate) ở mức nào và giữ được bao nhiêu trích dẫn hợp lệ cho câu hỏi lịch sử?</i>
RQ5	<i>Verification sáu tầng, đặc biệt numeric grounding, claim support và evidence completeness, làm giảm invalid citation và unsupported claim ở mức nào, đồng thời ảnh hưởng thế nào đến tỷ lệ abstention?</i>
RQ6	<i>Những nhóm lỗi nào giới hạn chất lượng hệ thống: parser, extraction, retrieval, temporal metadata, generation hay verification?</i>

1.5 Đối tượng và phạm vi

Đối tượng nghiên cứu là pipeline RAG phục vụ hỏi đáp pháp luật giao thông đường bộ Việt Nam. Corpus mục tiêu gồm khoảng 20 đến 30 văn bản chính thống, có cả văn bản hiện hành, lịch sử và ít nhất 5 chuỗi sửa đổi, thay thế hoặc bãi bỏ. Người dùng cuối có thể hỏi quy định hiện hành, hỏi tại ngày cụ thể hoặc so sánh hai thời điểm.

Đề tài không bao phủ toàn bộ hệ thống pháp luật, không cung cấp kết luận thay thế cơ quan có thẩm quyền, không tìm web tự do để sinh answer, không xây dựng multi-agent, không fine-tune LLM và không triển khai ứng dụng di động hoặc voice.

Bảng 1.2: Phân định phạm vi

Trong phạm vi	Ngoài phạm vi
PDF pháp luật giao thông đường bộ	Toàn bộ pháp luật Việt Nam
Parser Router: Docling chính, MinerU phụ/fallback	Crawler tự động activate tài liệu
Canonical Document IR và Legal Structure Extractor	Tư vấn pháp lý cá nhân hóa có tính kết luận
Quan hệ tham chiếu chéo trong PostgreSQL	Open web search làm nguồn answer
Temporal retrieval và sửa đổi từng phần	Autonomous multi-agent
Exact lookup + dense + sparse + RRF + reranker	Fine-tuning LLM
Evidence planning và Evidence Completeness Gate	Knowledge graph hoặc Neo4j
Verification sáu tầng và abstention	Mobile, voice, Kubernetes
Langfuse observability ngoài đường tới hạn	RAGFlow làm nền tảng chính
Gold set 200 câu và bốn suite A-D	Local LLM
RAGFlow làm baseline so sánh bên ngoài	Dịch vụ microservices

1.6 Phương pháp thực hiện

Nghiên cứu sử dụng phương pháp thiết kế và thực nghiệm hệ thống. Trước hết, yêu cầu và invariant pháp lý được phân tích để xây dựng data model. Tiếp theo, pipeline ingestion (Parser Router, Canonical Document IR, Legal Structure

Extractor, resolver), *retrieval* đa tầng và *verification* được cài đặt theo kiến trúc *module*. Sau đó, các biến thể được đánh giá trên cùng *corpus* và *gold set* qua bốn suite thí nghiệm A-D. Kết quả được phân tích theo *metric* xác định, *metric LLM* phụ, *latency*, *cost* và *error taxonomy*.

Quá trình thực hiện trải qua một lần thay đổi thiết kế: phương án thiết kế đầu tiên dựa trên một *framework* trích xuất dữ liệu phi cấu trúc tách rời [5]. Sau khi đánh giá, phương án này được thay bằng *Parser Router*, *Canonical Document IR* và *Legal Structure Extractor* do dự án sở hữu để giảm tầng chuyển đổi trung gian và kiểm soát trực tiếp việc nhận diện phân cấp pháp luật Việt Nam. Lý do chi tiết được trình bày trong Chương 2.

1.7 Đóng góp dự kiến

Sáu đóng góp chính gồm:

1. *Parser Router* định tuyến *Docling/MinerU* theo đặc tính tài liệu và *quality gate*, cùng *Canonical Document IR parser-neutral* do dự án sở hữu;
2. *Legal Structure Extractor* cho phân cấp pháp luật Việt Nam, gồm nhãn *Điểm tiếng Việt d*) và *đ*), *short-Point retention* và *stable ID* tái tạo được;
3. mô hình quan hệ tham chiếu chéo cấp *provision* và cấp văn bản lưu trong *PostgreSQL* kèm mở rộng ngữ cảnh pháp lý theo quan hệ;
4. *retrieval* đa tầng với *evidence planning*, *Evidence Completeness Gate* và mở rộng ngữ cảnh có ghi lý do (*added_by*, *depth*);
5. *verification* xác định sáu tầng với bất biến *Returned Invalid Citation Rate* bằng 0;
6. *gold set* tiếng Việt 200 câu và phương pháp *evaluation* bốn suite A-D có thể tái lập, kèm *baseline RAGFlow* bên ngoài.

1.8 Bố cục khóa luận

Chương 2 trình bày nền tảng lý thuyết và nghiên cứu liên quan. Chương 3 mô tả yêu cầu, *corpus*, *gold set* và phương pháp thực nghiệm. Chương 4 trình bày thiết kế hệ thống. Chương 5 mô tả cài đặt và triển khai. Chương 6 trình bày kiểm thử và đánh giá. Chương 7 tổng kết, nêu hạn chế và hướng phát triển.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1 Mô hình ngôn ngữ lớn

Transformer sử dụng cơ chế attention để mô hình hóa quan hệ giữa các token trong chuỗi và là nền tảng của nhiều mô hình ngôn ngữ hiện đại [6]. LLM có khả năng sinh văn bản, phân loại và trích xuất có cấu trúc, nhưng đầu ra không mặc nhiên là bằng chứng. Trong hệ thống này, LLM chỉ đảm nhiệm phân tích phần ngôn ngữ chưa xử lý được bằng rule và sinh answer từ context đã giới hạn.

Generator được cấu hình theo structured output. Gemini 3.5 Flash hỗ trợ structured outputs theo json_schema tương thích Pydantic và context lớn [7]. Tuy nhiên, model capability không thay thế retrieval correctness hoặc citation verification.

2.2 Retrieval-Augmented Generation

RAG kết hợp retriever với generator để cung cấp external knowledge cho mô hình [8]. Một hệ thống RAG có thể chia thành:

- 1. ingestion và indexing;*
- 2. query processing;*
- 3. retrieval;*
- 4. context construction;*
- 5. generation;*
- 6. verification và response rendering.*

Đề tài xem RAG như một hệ thống nhiều lớp. Parser sai làm extraction sai; extraction sai làm index sai; retrieval sai đặt trần cho generation; generation đúng nhưng citation sai vẫn không thể trả ra người dùng. Vì vậy, một pipeline RAG tổng quát (PDF, chunk, vector, LLM) là chưa đủ cho pháp luật: nó không biết ranh giới trích dẫn, không biết phiên bản hiệu lực, không biết quan hệ giữa các quy định và không kiểm chứng được câu trả lời.

2.3 Dense retrieval và embedding

Dense retrieval ánh xạ query và document sang vector thực. Độ tương đồng cosine thường được dùng để xếp hạng. Gemini Embedding 2 hỗ trợ output dimension linh hoạt từ 128 đến 3072 và khuyến nghị các mức 768, 1536 hoặc 3072

[9]. Hệ thống dùng 768 chiều làm cấu hình thử nghiệm khởi điểm để cân bằng chất lượng và chi phí lưu trữ, nhưng embedding production chưa được chốt vĩnh viễn: Suite B benchmark ba ứng viên (Gemini Embedding 2, Jina Embeddings v5 text-nano, Jina Embeddings v5 text-small) trên câu hỏi pháp luật tiếng Việt trước khi chọn.

Embedding text của một provision không chỉ chứa nội dung gốc mà còn có tên văn bản, số hiệu và parent context (retrieval_text). Việc bổ sung context giúp các Điểm ngăn vẫn mang đủ nghĩa khi được truy xuất độc lập, trong khi source_text luôn giữ nguyên văn bản pháp lý gốc.

2.4 Sparse retrieval và BM25

BM25 là mô hình xác suất dựa trên tần suất từ, độ dài tài liệu và độ hiếm của thuật ngữ [10]. Một dạng công thức phổ biến là:

$$BM25(q, d) = \sum_{t \in q} IDF(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{avgdl}\right)}. \quad (2.1)$$

Sparse retrieval đặc biệt hữu ích cho số hiệu, tên văn bản, cụm từ pháp lý và tham chiếu Điều/Khoản/Điểm. Hệ thống không loại mạnh các từ như ``không'', ``phải'', ``được'', ``trừ'', vì chúng có thể thay đổi nghĩa quy phạm. Qdrant cung cấp sparse/BM25 retrieval ngay trong collection bằng tokenizer-based model [11]; với tiếng Việt cần kiểm chứng tokenizer trong Suite C trước khi khóa cấu hình.

2.5 Hybrid retrieval và RRF

Dense và sparse score nằm trên scale khác nhau. Reciprocal Rank Fusion kết hợp các danh sách theo thứ hạng thay vì raw score [12]. Với document d và tập ranking R :

$$RRF(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}. \quad (2.2)$$

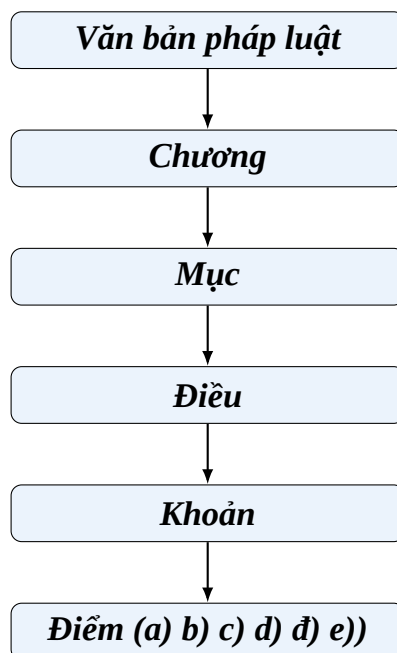
Qdrant hỗ trợ named dense/sparse vectors, prefetch và RRF trong Query API [13]. Điều này loại bỏ nhu cầu duy trì hai hệ retrieval tách rời và code fusion thủ công.

2.6 Chunking tài liệu

Chunking quyết định đơn vị được embed và retrieve. Fixed-token chunking đơn giản nhưng có thể phá cấu trúc. Docling cung cấp HierarchicalChunker và HybridChunker; HybridChunker thực hiện split chunk quá dài và merge peer chunk nhỏ dựa trên tokenizer, đồng thời giữ metadata của document structure [14].

Đối với pháp luật, hierarchy tổng quát chưa đủ. Một Điểm có thể phụ thuộc vào lead-in của Khoản; một Khoản có thể phụ thuộc tiêu đề Điều. Ranh giới chunk

phải trùng ranh giới trích dẫn. Vì vậy, *Legal Structure Extractor* của dự án xác định ranh giới pháp lý trước, còn *chunking* chỉ xử lý provision quá dài theo câu nhưng giữ cùng *provision_id*; *Docling chunker* không được dùng làm *legal structure parser* chính.



Hình 2.1: Cấu trúc phân cấp của một đơn vị pháp lý

2.7 Document parsing và provenance

Docling chuyển nhiều định dạng tài liệu sang một representation giàu cấu trúc, nhận biết layout và table, đồng thời hỗ trợ OCR tiếng Việt qua *Tesseract*, *EasyOCR* hoặc *RapidOCR* [15]. *MinerU* là parser phụ và *fallback/challenger* với OCR 109 ngôn ngữ và backend pipeline chạy CPU. Các khả năng chi tiết của hai parser được mô tả trong tài liệu nghiên cứu công nghệ của dự án; không khẳng định parser nào vượt trội tuyệt đối cho mọi trường hợp.

Hệ thống không đưa đầu ra parser trực tiếp vào phân tích pháp lý. Parser Router chọn parser theo đặc tính tài liệu và *quality gate*, sau đó đầu ra được chuyển sang *Canonical Document IR* do dự án sở hữu gồm *ParsedDocument*, *ParsedPage* và *DocumentElement*. Tầng IR cô lập *Legal Structure Extractor* khỏi định dạng đầu ra của *Docling/MinerU*; thay hoặc nâng cấp parser chỉ cần một adapter mới.

Provenance được giữ dưới dạng *page number*, *bounding box* và *source_element_ids* truy vết về IR. *Provenance* phục vụ ba mục tiêu: *review extraction*, hiển thị đoạn nguồn và audit khi answer bị phản ánh. Một citation không chỉ trỏ đến văn bản mà còn phải mở được passage cụ thể.

2.8 Temporal data và phiên bản pháp luật

Một provision hợp lệ tại ngày d khi:

$$\begin{aligned}
\text{valid}(p, d) = & (p.\text{effective_from} \leq d) \\
& \wedge (p.\text{effective_to} = \emptyset \vee d < p.\text{effective_to}) \\
& \wedge (p.\text{review_status} = \text{ACCEPTED}).
\end{aligned} \tag{2.3}$$

Khoảng thời gian được biểu diễn theo dạng nửa kín $[from, to)$. Khi một version mới có hiệu lực, version cũ kết thúc tại chính ngày bắt đầu của version mới. Partial amendment được xử lý ở mức provision thay vì đóng toàn bộ document: provision_id giữ nguyên, nội dung mới được lưu dưới version mới với khoảng hiệu lực mới. Các sự kiện pháp lý (có hiệu lực, sửa đổi, thay thế, bãi bỏ, đính chính) được ghi nhận bằng LegalEffectEvent để phục vụ temporal resolver và câu hỏi so sánh lịch sử.

Thời gian hiệu lực không phải metadata phụ mà là điều kiện lọc bắt buộc của mọi retrieval: câu trả lời đúng cho một hành vi năm 2023 phải dùng văn bản hợp lệ tại năm 2023, kể cả khi văn bản đó đã bị thay thế. Hệ thống không dùng văn bản hiện hành làm mặc định cho câu hỏi có thể là lịch sử.

2.9 Citation verification và abstention

LegalBench-RAG nhấn mạnh retrieval chính xác ở các đoạn pháp lý nhỏ thay vì trả cả document dài [16]. Legal RAG Bench tiếp tục chỉ ra retrieval là yếu tố chủ đạo trong hiệu năng end-to-end của legal RAG [17]. LegalCiteBench cho thấy citation generation không grounding là một failure mode nghiêm trọng [4].

Từ các kết quả này, hệ thống không yêu cầu LLM viết citation text. Mỗi claim chỉ được chọn provision_id từ whitelist context. Tuy nhiên, citation tồn tại là chưa đủ: một trích dẫn hợp lệ về định danh vẫn có thể đi kèm số liệu sai (mức phạt, số điểm trừ) hoặc claim không được passage hỗ trợ. Vì vậy, ngoài kiểm tra ID và hiệu lực, hệ thống còn kiểm tra numeric grounding và claim support. Abstention được coi là hành vi đúng khi evidence không đủ.

2.10 Evidence completeness

Nhiều câu hỏi pháp luật yêu cầu nhiều loại bằng chứng phối hợp. Ví dụ, câu hỏi về mức phạt và điểm trừ giấy phép lái xe cần cả monetary_penalty và license_points. Nếu pipeline chỉ truy xuất được mức phạt và trả lời luôn, người dùng nhận được câu trả lời nửa vời. Đề tài xử lý vấn đề này bằng evidence plan (danh sách loại bằng chứng cần thiết được tạo trong query understanding) và Evidence Completeness Gate trước khi sinh câu trả lời. Khi còn thiếu loại bằng chứng bắt buộc, hệ thống chạy targeted retrieval hoặc mở rộng ngữ cảnh theo quan hệ thay vì sinh câu trả lời.

Bằng chứng cho câu trả lời thường nằm rải rác ở nhiều provision liên quan. Mô hình quan hệ tham chiếu chéo (trình bày trong Chương 4) là cơ sở để mở rộng ngữ cảnh quanh các seed provision mạnh, đồng thời là dữ liệu gold cho metric

Cross-reference Resolution Recall và Multi-hop Evidence Completeness.

2.11 Đánh giá hệ thống RAG

RAGAS cung cấp các metric như *faithfulness*, *context precision* và *context recall* [18, 19]. Tuy nhiên, metric LLM có thể biến động theo judge và prompt. Vì vậy, khóa luận dùng metric xác định làm headline (*Recall@k*, *MRR*, *nDCG*, *Citation P/R/F1*, *Temporal Validity Accuracy*, *Numeric Grounding Accuracy*, *Evidence Set Recall*, *Abstention P/R/F1*), còn Ragas và judge độc lập GPT-5.4 mini chỉ bổ sung đánh giá semantic [20].

2.12 Khoảng trống nghiên cứu

Các benchmark pháp luật hiện có chủ yếu dùng tiếng Anh, hợp đồng hoặc án lệ. Đề tài tập trung vào văn bản quy phạm pháp luật giao thông Việt Nam, nơi hierarchy, nhãn Điểm tiếng Việt và hiệu lực theo thời gian là những đặc điểm bắt buộc. Khoảng trống cụ thể gồm:

- chưa có benchmark nhỏ, review được cho current/historical/comparison query tiếng Việt;
- pháp luật Việt Nam chưa được đánh giá trên sự khác biệt giữa parser tổng quát và parser pháp lý chuyên biệt;
- citation ID, temporal validity và numeric grounding thường chưa được kiểm chứng cùng lúc;
- nhiều pipeline trộn web fallback vào answer path, làm giảm khả năng tái lập;
- câu hỏi đa bằng chứng và quan hệ tham chiếu chéo chưa được đánh giá như hạng mục đầu tiên trong legal RAG benchmark.

Bảng 2.1: So sánh định hướng nghiên cứu liên quan

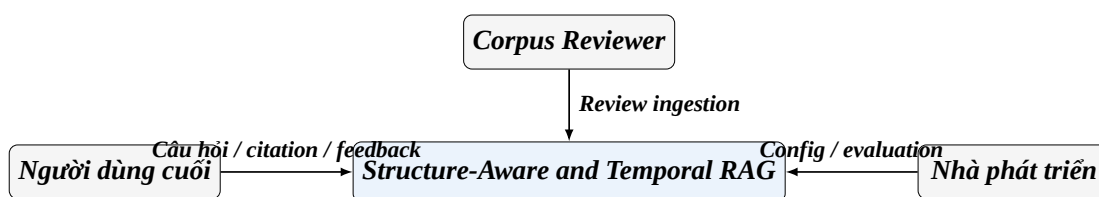
Công trình	Retrieval	Citation	Temporal	Định hướng chính
RAGAS	Có metric	Gián tiếp	Không	Đánh giá RAG tổng quát
LegalBench-RAG	Chính xác	Không là trọng tâm	Không	Retrieval đoạn pháp lý nhỏ
Legal RAG Bench	End-to-end	Độ bám nguồn	Hạn chế	Phân rã lỗi retrieval/reasoning
LegalCiteBench	Closed-book	Trọng tâm	Không	Citation recovery và verification
Đề tài này	Exact dense sparse	+ ID + sáu tầng verifier	Có	Luật giao thông Việt Nam

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU VÀ PHƯƠNG PHÁP NGHIÊN CỨU

3.1 Tác nhân và use case

Ba tác nhân chính là người dùng cuối, corpus reviewer và nhà phát triển. Người dùng hỏi luật, xem citation và gửi feedback. Reviewer kiểm tra metadata, hierarchy, provenance, quan hệ và hiệu lực trước khi index. Nhà phát triển quản lý corpus, model, deployment, evaluation và baseline RAGFlow. HITL chỉ nằm ở khâu review trong ingestion pipeline, không nằm trong online query.



Hình 3.1: Các tác nhân chính của hệ thống

Hệ thống có mười hai use case: UC-01 hỏi luật hiện hành, UC-02 hỏi luật lịch sử, UC-03 so sánh quy định, UC-04 tìm provision, UC-05 xem citation và nguồn, UC-06 abstention, UC-07 ingest tài liệu bất đồng bộ, UC-08 review ingestion, UC-09 chạy evaluation, UC-10 gửi feedback, UC-11 chạy parser benchmark (Suite A) và UC-12 corpus QA review.

3.2 Yêu cầu chức năng

Bảng 3.1: Yêu cầu chức năng P0 (rút gọn)

Mã	Yêu cầu	Tiêu chí kiểm chứng
FR-01	Parser Router (Docling MinerU)	Routing đúng theo đặc tính tài liệu; quality gate kích hoạt fallback
FR-02	Canonical Document IR parser-neutral	Extractor chỉ đọc IR; adapter mới không đổi extractor
FR-03	Legal Structure Extractor	Phân cấp đúng; nhãn d)/đ) không lẫn; giữ Điểm ngăn
FR-04	Parent-context enrichment	source_text bắt biến; retrieval_text kế thừa context cha

<i>Mã</i>	<i>Yêu cầu</i>	<i>Tiêu chí kiểm chứng</i>
<i>FR-05</i>	<i>Legal Reference Resolver</i>	<i>Quan hệ khớp fixture; unresolved định tuyến review</i>
<i>FR-06</i>	<i>Temporal and Amendment Resolver</i>	<i>Interval đúng quy tắc; hiệu lực không chắc chắn định tuyến review</i>
<i>FR-07</i>	<i>Background ingestion qua hàng đợi</i>	<i>Upload trả 202; actor idempotent; không parse đồng bộ</i>
<i>FR-08</i>	<i>Object storage qua MinIO</i>	<i>Round-trip put/get; backup sang nơi độc lập</i>
<i>FR-09</i>	<i>Review routing trước khi index</i>	<i>Chỉ accepted được index; dropped không bao giờ index</i>
<i>FR-10</i>	<i>Corpus QA</i>	<i>Báo cáo đủ 16 chỉ số</i>
<i>FR-11</i>	<i>Query Understanding và evidence plan</i>	<i>Intent/date/refs đúng; evidence plan liệt kê đủ loại bằng chứng</i>
<i>FR-12</i>	<i>Query Expansion</i>	<i>Giữ câu gốc; rewrite có giới hạn; HyDE chỉ bật có điều kiện</i>
<i>FR-13</i>	<i>Exact legal lookup</i>	<i>Trả đúng provision theo số hiệu, Điều, Khoản, Điểm</i>
<i>FR-14</i>	<i>Dense + sparse + RRF</i>	<i>Ba kênh chạy độc lập; variant tái lập bằng config</i>
<i>FR-15</i>	<i>Reranking</i>	<i>Chạy như stage chuẩn; không khẳng định cải thiện trước benchmark</i>
<i>FR-16</i>	<i>Legal context expansion</i>	<i>Chỉ mở rộng quanh seed mạnh; ghi added_by và depth</i>
<i>FR-17</i>	<i>Evidence Completeness Gate</i>	<i>Câu đa bằng chứng thiếu loại bằng chứng bị đánh dấu INCOMPLETE</i>
<i>FR-18</i>	<i>Hỏi luật hiện hành</i>	<i>Chỉ cite provision có hiệu lực tại ngày hỏi</i>
<i>FR-19</i>	<i>Hỏi luật lịch sử</i>	<i>Dùng phiên bản hợp lệ tại mốc hỏi, không dùng hiện hành làm mặc định</i>

<i>Mã</i>	<i>Yêu cầu</i>	<i>Tiêu chí kiểm chứng</i>
<i>FR-20</i>	<i>So sánh hai thời điểm</i>	<i>Hai temporal contexts độc lập; không gộp citation hai mốc</i>
<i>FR-21</i>	<i>Tìm kiếm provision</i>	<i>Trả top-k kèm hierarchy, hiệu lực và provenance; không gọi generator</i>
<i>FR-22</i>	<i>Structured generation</i>	<i>Output đúng schema cấp claim; chỉ dùng ID trong whitelist</i>
<i>FR-23</i>	<i>Verification sáu tầng L1-L6</i>	<i>Invalid citation không trả ra; claim số liệu sai bị chặn</i>
<i>FR-24</i>	<i>Failure-aware repair và abstention</i>	<i>Mỗi lỗi có đường sửa riêng; hết giới hạn thì ABSTAIN</i>
<i>FR-25</i>	<i>Disclaimer</i>	<i>Có trong mọi verified/abstained response</i>
<i>FR-26</i>	<i>Observability qua Langfuse</i>	<i>Trace ghi lại được; Langfuse down không fail query</i>
<i>FR-27</i>	<i>Feedback người dùng cuối</i>	<i>Round-trip; gắn đúng trace_id; danh mục báo cáo đầy đủ</i>
<i>FR-28</i>	<i>Evaluation và ablation</i>	<i>Bốn suite A-D chạy được; run lưu đủ metadata</i>
<i>FR-31</i>	<i>Benchmark đối chiếu RAGFlow</i>	<i>Bốn variant trên cùng corpus và eval queries; chạy riêng</i>
<i>FR-32</i>	<i>Hiển thị answer và citation</i>	<i>Citation dựng từ metadata; không stream draft chưa verify</i>

Các yêu cầu FR-29 (lưu lịch sử truy vấn) và FR-30 (admin review UI) là P1, chỉ triển khai sau khi mọi acceptance criteria P0 đạt; trong P0, reviewer dùng CLI và JSON report.

3.3 Yêu cầu phi chức năng

Các invariant quan trọng gồm: returned invalid citation rate bằng 0; unverified answer không được trả; mọi indexed provision có page provenance và thuộc document đã accept; final evaluation phải lưu corpus hash và gold-set hash; hệ thống chạy local bằng Docker Compose với bảy service. Các ngưỡng hiệu năng là mục tiêu kỹ thuật trong môi trường test, không phải kết quả đo: retrieval P95 không quá 2 giây, end-to-end P50 không quá 12 giây, end-to-end P95 không quá 20 giây, search API P95 không quá 3 giây. Ingestion chạy nền với

MAX_INGESTION_WORKERS=1. Langfuse không nằm trên đường tới hạn; nếu không khả dụng, query vẫn hoạt động.

3.4 Thiết kế corpus

Corpus được chọn theo bốn tiêu chí:

1. *nguồn chính thống và có thể kiểm tra lại;*
2. *liên quan giao thông đường bộ;*
3. *có metadata ban hành và hiệu lực;*
4. *tạo được nhóm current, historical và relation chain.*

Quy mô mục tiêu là 20 đến 30 văn bản chính thống, có ít nhất 5 chuỗi sửa đổi, thay thế hoặc bãi bỏ, có cả văn bản hiện hành và lịch sử. Mỗi file có manifest chứa document ID, số hiệu, ngày ban hành, khoảng hiệu lực, URL nguồn, file hash, relation notes và review status. Ưu tiên PDF born-digital có text layer; tài liệu scan đi qua OCR backend CPU và phải qua quality gate trước khi review.

Listing 3.1: Ví dụ corpus manifest rút gọn

```
1 {  
2   "document_id": "nd-168-2024",  
3   "document_number": "168/2024/ND-CP",  
4   "document_type": "DECREE",  
5   "issued_date": "2024-12-26",  
6   "effective_from": "2025-01-01",  
7   "effective_to": null,  
8   "source_url": "https://vbpl.vn/...",  
9   "file_hash": "sha256:...",  
10  "relation_notes": "SUPERSEDES nd-100-2019",  
11  "review_status": "ACCEPTED"  
12 }
```

Corpus có báo cáo/dashboard chất lượng riêng (corpus QA) với các chỉ số: document count, article count, clause count, point count, Point coverage, short-Point retention, tỷ lệ phát hiện nhãn đ), orphan Point count, orphan Clause count, duplicate provision count, parent-context coverage, provenance coverage, table coverage, unresolved cross-reference count, unknown effective date count và temporal conflict count. Với các văn bản quan trọng (ví dụ Nghị định 168), thực hiện structural QA có mục tiêu riêng.

3.5 Thiết kế gold set

Gold set mục tiêu gồm 200 câu đã review, chia 40 development (lập phát triển), 40 validation (chọn ngưỡng/model/prompt) và 120 final test (báo cáo kết quả, đóng băng, không dùng để tuning). Có 17 danh mục bắt

buộc: *CURRENT, HISTORICAL, COMPARISON, EXACT_REFERENCE, PENALTY, LICENSE_POINTS, CONDITION, EXCEPTION, PROCEDURE, CROSS_REFERENCE, MULTI_PROVISION, MULTI_DOCUMENT, COLLOQUIAL_QUERY, AMBIGUOUS, MISSING_INFORMATION, OUT_OF_SCOPE, ADVERSARIAL_CITATION.*

Mỗi record chứa *id, question, category, query_date, comparison_dates, vehicle_type, expected_provision_ids, acceptable_provision_ids, required_evidence, must_include_facts, must_not_include_facts, temporal_metadata, review_status, reviewed_by, gold_version* và *hash*. Mỗi câu có thể yêu cầu nhiều provision bằng chứng; *required_evidence* liệt kê các loại bằng chứng như *violation_definition, monetary_penalty, license_points, license_suspension, exception, procedure, legal_condition*.

3.6 Thiết kế thí nghiệm

Bốn suite thí nghiệm thay thế mô hình ablation đơn lẻ của thiết kế trước:

Bảng 3.2: Suite A - Parser benchmark

Thí nghiệm	Parser
P1	Docling
P2	MinerU
P3	Parser Router (quyết định routing + quality gates)

Chỉ số Suite A: Article P/R/F1, Clause P/R/F1, Point P/R/F1, Short Point Recall, Vietnamese đ) Recall, Parent Context Completeness, Table Preservation, Header/Footer Leakage, Provenance Coverage.

Bảng 3.3: Suite B - Embedding benchmark

Thí nghiệm	Model	Dimensions
E1	Gemini Embedding 2	768
E2	Jina Embeddings v5 text-nano	768
E3	Jina Embeddings v5 text-small	1024

Chỉ số Suite B: Recall@10, MRR@10, nDCG@10 trên câu hỏi pháp luật tiếng Việt, latency, cost.

Bảng 3.4: Suite C - Retrieval ablation

Thí nghiệm	Cấu hình
R1	<i>Legal chunk + dense</i>
R2	<i>R1 + sparse/RRF</i>
R3	<i>R2 + query normalization</i>
R4	<i>R3 + multi-query rewrite</i>
R5	<i>R4 + conditional HyDE</i>
R6	<i>R5 + reranker</i>
R7	<i>R6 + parent/sibling expansion</i>
R8	<i>R7 + cross-reference expansion</i>
R9	<i>R8 + temporal filtering</i>
R10	<i>Complete retrieval pipeline</i>

Bảng 3.5: Suite D - Generation và verification ablation

Thí nghiệm	Cấu hình
G1	<i>Prompt-only</i>
G2	<i>Structured output</i>
G3	<i>G2 + citation ID verifier</i>
G4	<i>G3 + temporal verifier</i>
G5	<i>G4 + numeric grounding</i>
G6	<i>G5 + claim support</i>
G7	<i>G6 + evidence completeness</i>

Ma trận thí nghiệm được thiết kế có tham khảo các quan sát bên ngoài từ dự án Traffic-RAG (ranh giới chunk trùng ranh giới trích dẫn, Điểm ngăn không được lọc bỏ, ấ) phải được nhận diện, retrieval cần ngữ cảnh câu mở đầu Khoản, mở rộng Khoản lân cận, citation filtering đơn thuần là chưa đủ, label gold có thể sai cần review độc lập). Các quan sát này chỉ là động lực thiết kế thí nghiệm, không phải kết quả của VNLrag.

3.7 Metric

Với gold set G_q và top-k result R_q^k :

$$\text{Recall@k}(q) = \frac{|G_q \cap R_q^k|}{|G_q|}. \quad (3.1)$$

Nếu relevant item đầu tiên ở rank r_q :

$$MRR = \frac{1}{N} \sum_{q=1}^N \frac{1}{r_q}. \quad (3.2)$$

Citation Precision, Recall và F1 được tính bằng exact/acceptable provision IDs. Temporal Validity Accuracy đo tỷ lệ citation hợp lệ tại query date; Temporal Leakage Rate đo tỷ lệ citation dùng phiên bản sai thời điểm. Abstention được đánh giá bằng precision, recall, F1 và confusion matrix.

Bảng 3.6: Nhóm metric đánh giá

<i>Lớp</i>	<i>Metric</i>
<i>Corpus</i>	<i>Hierarchy F1, Point Coverage, Short Point Recall, Provenance Coverage, Parent Context Coverage</i>
<i>Retrieval</i>	<i>Recall@5/10/20, MRR@10, nDCG@10</i>
<i>Evidence</i>	<i>Evidence Set Recall, All Required Evidence@10, Cross-reference Resolution Recall, Multi-hop Evidence Completeness</i>
<i>Temporal</i>	<i>Temporal Validity Accuracy, Temporal Leakage Rate, Current/Historical Separation Accuracy, Comparison Separation Accuracy</i>
<i>Citation</i>	<i>Precision, Recall, F1, Invalid Citation Rate</i>
<i>Grounding</i>	<i>Numeric Grounding Accuracy, Unsupported Claim Rate, Claim Support Precision, Answer Evidence Completeness</i>
<i>Abstention</i>	<i>Precision, Recall, F1</i>
<i>Vận hành</i>	<i>P50/P95 latency, token usage, cost, parser time, indexing time</i>

3.8 Chống leakage và bảo đảm tái lập

Final test set được đóng băng trước final run và không bao giờ dùng để tuning. Thay đổi gold label phải tạo version mới, không sửa file đã đóng băng. Mỗi run lưu run_id, git commit, corpus version/hash, gold-set version/hash, suite và variant, run_manifest_hash, model IDs, prompt versions, parser versions, config snapshot, token usage, estimated cost và raw results path. Run và raw artifact bất biến/append-only; mọi query fail vẫn được giữ trong error analysis.

3.9 Đề dọa đến tính hợp lệ

Các đề dọa gồm corpus nhỏ và một reviewer chính có thể tạo annotation bias, LLM judge variance, provider model drift, query distribution chưa đại diện toàn bộ người dùng, và khả năng label gold sai. Biện pháp giảm thiểu là dùng

metric xác định làm headline, pin model snapshot, lưu raw output, review độc lập label gold, chia data split rõ ràng và công khai limitation.

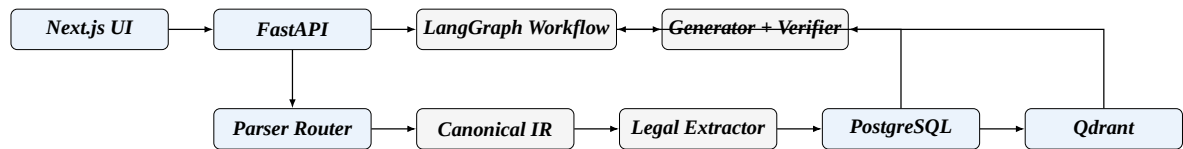
CHƯƠNG 4

THIẾT KẾ HỆ THỐNG

4.1 Nguyên tắc thiết kế

Hệ thống tuân theo các nguyên tắc: *parser-neutral IR*, *PostgreSQL* là nguồn chân lý, *Qdrant* là index dẫn xuất dựng lại được, *temporal-by-default*, *citation-by-ID*, *verified-or-abstain*, *structure-aware*, chất lượng parser là mục tiêu hàng nhất, quan hệ tham chiếu được mô hình hóa tường minh, *evidence completeness* trước khi sinh câu trả lời, *verification deterministic-first*, *failure-aware repair* có giới hạn, *controlled workflow* (không *autonomous agent*), *Langfuse* ngoài đường tới hạn, *RAGFlow* chỉ là baseline bên ngoài, không dùng *open-web search*, *reproducible experiments* và *local-first defense*.

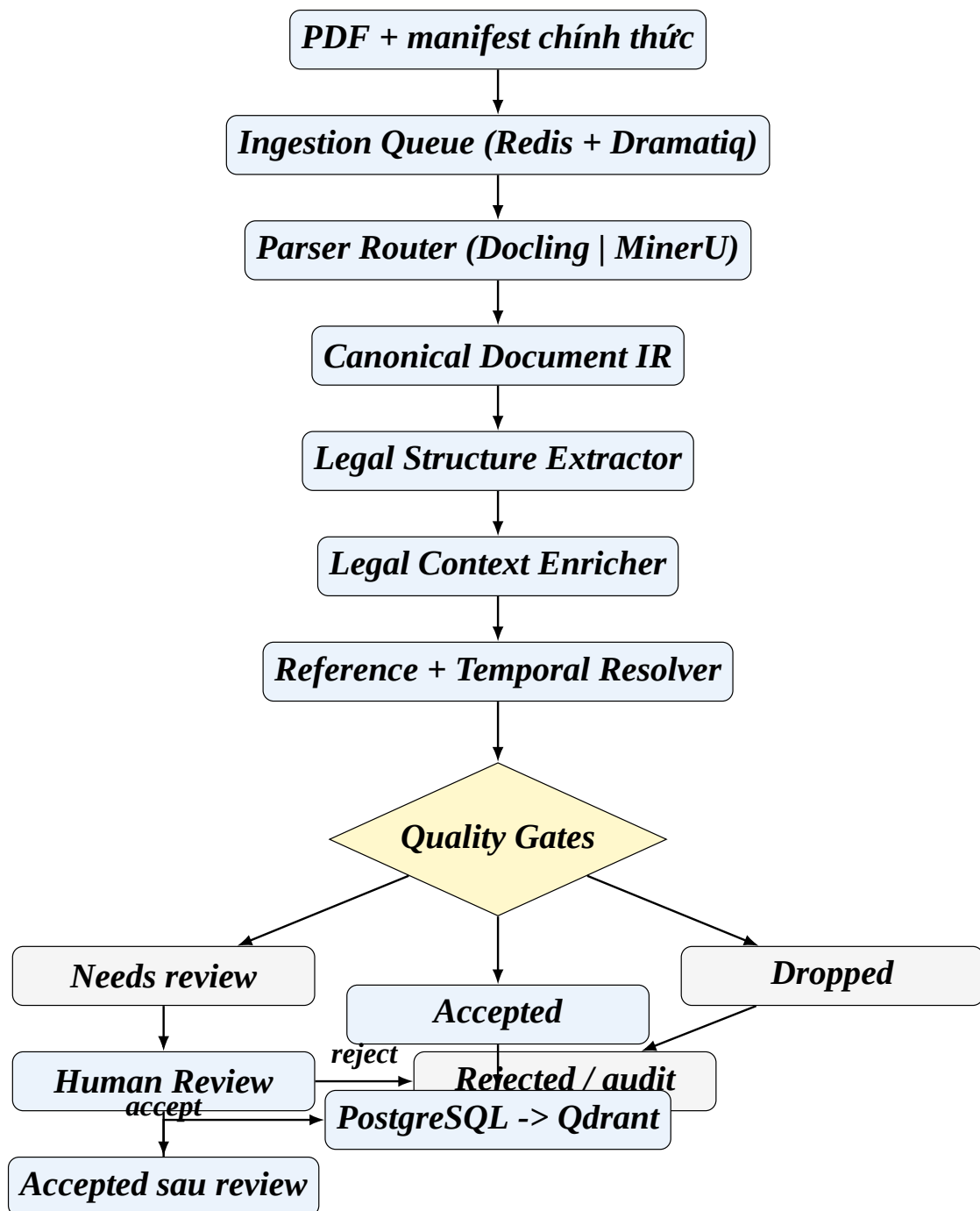
4.2 Kiến trúc tổng quan



Hình 4.1: Kiến trúc logic tổng thể của hệ thống

Hệ thống gồm hai pipeline tách biệt: *offline ingestion* và *online query*. *Observability* (*Langfuse*) chạy xuyên suốt nhưng không nằm trên đường tới hạn tính đúng đắn.

4.3 Pipeline ingestion ngoại tuyến



Hình 4.2: Pipeline xử lý và review tài liệu

Pipeline worker thực hiện parse -> normalize -> legal extract -> reference resolve -> temporal resolve -> quality gates -> review -> embed -> index, mỗi bước là một actor Dramatiq ngắn, rời rạc và idempotent. Chỉ kết quả phân loại accepted mới được index; needs_review phải qua quyết định reviewer; dropped chỉ được ghi audit và không bao giờ index. Qdrant chỉ nhận dữ liệu có review_status = ACCEPTED đọc từ PostgreSQL.

4.4 Parser Router và Canonical Document IR

Parser Router quyết định parser theo đặc tính tài liệu và quality gate (FR-01). Docling là parser chính; MinerU là parser phụ và fallback/challenger. Quy tắc routing:

- *PDF searchable có text layer và layout chuẩn: Docling chạy trước;*
- *PDF scan hoặc layout lỗi: Docling chạy trước với OCR backend CPU, nếu quality gate fail thì chuyển MinerU;*
- *bảng phức tạp: so sánh đầu ra hai parser khi cần.*

Quality gate chia hai nhóm: nhóm A (parser-level) chạy sau khi chuẩn hóa IR với các chỉ số như provenance coverage, text extraction rate, table detection và layout coherence; nhóm B (structural) chạy sau Legal Structure Extractor với các chỉ số như point label detection, hierarchy completeness và short-Point retention. Nếu nhóm structural fail, kết quả structural hiện tại bị hủy và Router chạy lại toàn bộ pipeline từ parser thay thế, không trộn kết quả hai parser. Mọi quyết định routing và kết quả quality gate được ghi vào ingestion_runs.parser_routing và DocumentElement.source_parser.

Canonical Document IR là biểu diễn trung gian do dự án sở hữu, gồm ParsedDocument chứa ParsedPage[], mỗi trang chứa DocumentElement[]. Mỗi element mang element_id, element_type, text, page_number, bbox, reading_order, parent_element_id, table_html, source_parser, parser_version, parser_confidence và raw_reference. Tầng IR cô lập phân tích pháp lý khỏi định dạng đầu ra của Docling/MinerU.

4.5 Legal Structure Extractor

Legal Structure Extractor là parser pháp lý riêng của VNLRAG, chạy trên Canonical Document IR, nhận diện Chương, Mục, Điều, Khoản, Điểm, Phụ lục, bảng pháp lý, điều khoản chuyển tiếp, tiêu đề và đánh số văn bản pháp luật Việt Nam kèm biến thể do OCR.

Điểm nhấn quan trọng nhất là nhãn Điểm tiếng Việt: bắt buộc hỗ trợ a) b) c) d) đ) e) theo bảng chữ cái tiếng Việt 29 ký tự, không dùng giả định [a - z]. Ánh xạ sang provision ID giữ nguyên ký tự tiếng Việt: Điểm d) ứng diem-d, Điểm đ) ứng diem-đ, tránh va chạm ID. Khi OCR không phân biệt được d) và đ), extractor dùng pattern ngữ cảnh; nếu không đủ ngữ cảnh thì gắn cờ ambiguity và định tuyến needs_review, không tự chọn bừa.

Extractor cũng bảo đảm short-Point retention: một Điểm pháp lý ngắn nhưng hợp lệ vẫn là provision hợp lệ, không bị loại bỏ vì số token thấp. Quy tắc tạo provision ID:

`{loai-van-ban}-{so}-{nam}__dieu-{n}__khoan-{n}__diem
-{chu-cai}`

Ví dụ: `nd-168-2024__dieu-7__khoan-4__diem-b`. Các *node* không thuộc cây Điều thường dùng dạng `__phu-luc-{n}`, `__phu-luc-{n}__bang-{m}`, `__dieu-{n}__bang-{m}`, `__dieu-{n}__khoan-chuyen-tiep`, `__chuyen-tiep-{k}`, `__tieu-de-{n}`.

Legal Context Enricher bổ sung *parent-context* vào *retrieval_text* (câu mở đầu của Khoản, tiêu đề Điều) để phục vụ *retrieval*; *source_text* không bao giờ bị biến đổi. Trích dẫn vẫn trỏ tới *provision thực tế*. Ví dụ minh họa: *source_text* là `p) Dàn hàng ngang từ 03 xe trở lên`; *retrieval_text* là `Khoản 4. Phạt tiền từ ... đến ... đối với một trong các hành vi sau: p) Dàn hàng ngang từ 03 xe trở lên`. Các con số trong ví dụ là *placeholder* minh họa, không phải khẳng định nội dung văn bản thực tế.

4.6 Mô hình dữ liệu

4.6.1 LegalDocument và DocumentVersion

LegalDocument quản lý *document ID*, số hiệu, tiêu đề, loại văn bản, cơ quan ban hành, ngày ban hành, nguồn, file hash và trạng thái. *DocumentVersion* ghi nhận mỗi phiên bản nội dung với *manifest bất biến*, hash và khoảng hiệu lực, gắn với các quan hệ pháp lý.

4.6.2 LegalProvision

Listing 4.1: Mô hình LegalProvision rút gọn

```
1 class LegalProvision(BaseModel):
2     provision_id: str
3     document_version_id: str
4     node_kind: str                # ARTICLE/CLAUSE/POINT/
        APPENDIX/TABLE/...
5     chapter: str | None = None
6     section: str | None = None
7     article: str | None = None
8     clause: str | None = None
9     point: str | None = None
10    heading: str | None = None
11    source_text: str              # bat bien, giu nguyen van ban
        goc
12    retrieval_text: str           # co the ke thua
        parent_context
13    parent_context: str | None = None
14    effective_from: date | None = None
15    effective_to: date | None = None
16    status: str
17    page_number: int
```

```

18     bbox: BoundingBox | None = None
19     source_element_ids: list[str]
20     content_hash: str
21     version: int
22     review_status: str # PENDING/ACCEPTED/REJECTED/
        DROPPED

```

`source_text` là nội dung pháp lý thuộc trực tiếp *provision* và giữ nguyên; `retrieval_text` phục vụ *retrieval* và có thể kế thừa ngữ cảnh cha. `provision_id` ổn định và tái tạo được; khi *provision* bị sửa đổi, ID giữ nguyên để trích dẫn ổn định, nội dung mới lưu dưới *version* mới. Unique key vật lý là cặp (`provision_id`, `version`). `status` mô tả vòng đời pháp lý của *provision*; `review_status` mô tả trạng thái review corpus và là cổng chặn trong điều kiện hiệu lực; hai trường này độc lập nhau.

4.6.3 *ProvisionReference* và *DocumentRelation*

ProvisionReference ghi quan hệ cấp *provision* với bốn loại: PARENT_OF, REFERS_TO, SIBLING_OF, PENALTY_COMPANION. Trong đó PENALTY_COMPANION gắn quy định xử phạt với quy định đi kèm như trừ điểm giấy phép lái xe hoặc tước quyền sử dụng giấy phép lái xe. *DocumentRelation* ghi quan hệ cấp văn bản với sáu loại: AMENDS, REPEALS, SUPERSEDES, CORRECTS, GUIDES, RELATED_TO. Quan hệ được *version-bound* (`source_provision_version_id`, `target_provision_version_id`) và lưu trong bảng PostgreSQL, xử lý bằng *application logic*; không dùng Neo4j hoặc *knowledge graph*. Reference không giải quyết được ghi UNRESOLVED/PENDING_REVIEW và định tuyến review, không suy đoán quan hệ.

4.6.4 *LegalEffectEvent*

LegalEffectEvent ghi lại sự kiện pháp lý ảnh hưởng tới hiệu lực của văn bản/*provision*: EFFECTIVE, AMENDED, PARTIAL_AMENDED, SUPERSEDED, REPEALED, CORRECTED, EXPIRED. `affected_provision_versions` liệt kê structured các *provision/version* chịu ảnh hưởng. Sự kiện này phục vụ *Temporal and Amendment Resolver* và câu hỏi so sánh lịch sử.

4.7 PostgreSQL schema

PostgreSQL là nguồn chân lý, lưu toàn bộ metadata, phiên bản, quan hệ, review, audit, query trace và feedback [21].

Bảng 4.1: Các bảng dữ liệu chính

Bảng	Vai trò
<i>legal_sources</i>	<i>Nguồn văn bản chính thống và mức ưu tiên đối chiếu</i>
<i>legal_documents</i>	<i>Document và metadata</i>
<i>document_versions</i>	<i>Phiên bản văn bản, manifest, hash và khoảng hiệu lực</i>
<i>legal_provisions</i>	<i>Provision version đầy đủ: hierarchy, text, interval, review</i>
<i>provision_versions</i>	<i>Registry lineage, superseded_by_version</i>
<i>provision_references</i>	<i>Quan hệ cấp provision, version-bound</i>
<i>document_relations</i>	<i>Quan hệ cấp văn bản (AMENDS, SUPERSEDES, REPEALS, ...)</i>
<i>legal_effect_events</i>	<i>Sự kiện pháp lý ảnh hưởng hiệu lực</i>
<i>parsed_documents</i>	<i>/ Canonical Document IR đã chuẩn hóa</i>
<i>document_elements</i>	
<i>ingestion_runs</i>	<i>Trạng thái job và parser routing</i>
<i>ingestion_artifacts</i>	<i>Object key trong MinIO và metadata</i>
<i>review_items</i>	<i>Bảng chứng và quyết định reviewer</i>
<i>query_traces</i>	<i>/ Lịch sử query, verification summary và feedback</i>
<i>query_feedback</i>	
<i>evaluation_datasets</i>	<i>Gold set, run metadata và raw result</i>
<i>/ evaluation_runs</i>	
<i>evaluation_results</i>	
<i>corpus_qa_reports</i>	<i>Báo cáo chất lượng corpus</i>

Các ràng buộc quan trọng: *UNIQUE (provision_id, version); CHECK interval effective_to IS NULL OR effective_to > effective_from; CHECK review-required review_status <> 'ACCEPTED' OR effective_from IS NOT NULL; exclusion constraint daterange ngăn hai version ACCEPTED chồng lấn trong cùng provision. Alembic quản lý migration; Qdrant dựng lại được từ SELECT * FROM legal_provisions WHERE review_status = 'ACCEPTED'.*

4.8 Qdrant collection

Mỗi Qdrant point đại diện cho một provision version *ACCEPTED*. Collection có named dense vector *dense* (768 chiều khởi điểm, Cosine) và sparse vector *sparse* (BM25). Payload chứa provision ID, provision version,

document ID/version, number, type, hierarchy fields, vehicle types, interval, review status, parser version, legal parser version, sparse encoder version, content hash và relation metadata giới hạn (REFERS_TO/PENALTY_COMPANION đã RESOLVED và ACCEPTED). Qdrant hỗ trợ hybrid query, RRF và payload filter trực tiếp [13].

Collection được version hóa, ví dụ legal_provisions_v1. Alias legal_provisions_active trỏ đến version đang phục vụ. Khi đổi embedding, sparse encoder, payload schema hoặc chunking production, hệ thống tạo collection mới, chạy regression rồi switch alias; không bao giờ rebuild in place và không trộn hai embedding space.

4.9 Retrieval đa tầng

4.9.1 Query Understanding và temporal resolution

Query Understanding ưu tiên deterministic parsing bằng regex cho ngày, năm, số hiệu văn bản, Điều/Khoản/Điểm và loại phương tiện; LLM structured extraction chỉ xử lý phần còn lại. Output là QueryPlan gồm intent, effective date, comparison dates, vehicle type, document number, article/clause/point, legal entities, normalized query, required evidence (evidence plan) và missing query information. Các intent gồm CURRENT, HISTORICAL, COMPARISON, SOURCE_SEARCH và OUT_OF_SCOPE.

Chính sách ngày: không có ngày thì dùng ngày request; có ngày cụ thể thì dùng ngày đó; chỉ có năm và không có sự kiện đổi hiệu lực trong năm thì áp dụng canonical date (ví dụ 01/07 của năm) và bắt buộc hiển thị ngày đã áp dụng; có sự kiện đổi hiệu lực trong năm thì yêu cầu ngày cụ thể hoặc ABSTAIN với MISSING_QUERY_DATE. Hệ thống không dùng văn bản hiện hành làm mặc định cho câu hỏi lịch sử.

4.9.2 Query Expansion

Query expansion luôn giữ câu hỏi gốc của người dùng. Các variant gồm original, normalized (chuẩn hóa thuật ngữ pháp lý như "phạt bao nhiêu" sang "mức xử phạt"), multi-query rewrite (giới hạn, không đệ quy) và conditional HyDE. HyDE chỉ bật khi câu ngắn, khẩu ngữ, ngữ nghĩa yếu hoặc bằng chứng chưa đủ, không bật luôn và không dùng cho exact lookup hay sparse.

4.9.3 Ba kênh retrieval, RRF và reranking

Ba kênh chạy song song:

1. exact legal lookup theo payload filter và SQL cho định danh như 168/2024/NĐ-CP, Điều 7, Khoản 4, Điểm a;
2. dense semantic với named vector dense;
3. sparse lexical BM25 với named vector sparse.

Dense và sparse được hợp nhất bằng RRF (k và weights cấu hình được), sau đó post-filter temporal invariant và dedup theo provision_id; các candidate exact lookup được bảo toàn và ghép vào tập đã fusion theo policy rõ ràng (exact match được ưu tiên giữ lại, loại trùng lặp theo provision_id) trước khi rerank. Reranking dùng Jina Reranker v3 như stage chuẩn của pipeline, nhưng không khẳng định reranker cải thiện chất lượng trước khi có kết quả R6 của Suite C.

4.9.4 Legal context expansion

Mở rộng ngữ cảnh chỉ quanh các seed provision mạnh theo parent, sibling, direct cross-reference và penalty companion, cùng các quy định liên quan sửa đổi/thay thế. Mỗi provision mở rộng ghi lý do vào context:

Listing 4.2: Metadata của provision được mở rộng

```

1 {
2   "provision_id": "nd-168-2024__dieu-7__khoan-4__diem-b",
3   "added_by": "CROSS_REFERENCE",
4   "source_id": "nd-168-2024__dieu-33__khoan-2",
5   "depth": 1
6 }
```

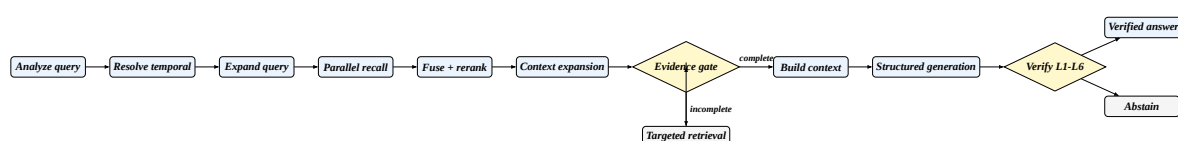
Mở rộng theo quan hệ phải áp dụng temporal filter và review filter theo ngày query; quan hệ PENDING_REVIEW/UNRESOLVED không được dùng để mở rộng tự động; depth có giới hạn để tránh mở rộng đồ thị vô hạn.

4.10 Evidence Completeness Gate

Trước khi sinh câu trả lời, Evidence Completeness Gate kiểm tra mọi loại bằng chứng trong evidence plan đã được thu thập trong context. Nếu trạng thái là INCOMPLETE (ví dụ câu hỏi mức phạt + điểm trừ mà chỉ tìm được mức phạt), hệ thống chạy targeted retrieval theo evidence_gaps hoặc mở rộng ngữ cảnh theo quan hệ, rồi kiểm tra lại trước khi gọi generator. Mỗi lượt quay lại tính vào repair_attempts hữu hạn; hết giới hạn thì ABSTAIN. Hệ thống không âm thầm trả lời một nửa để của câu hỏi đa bằng chứng.

4.11 Workflow hỏi đáp

LangGraph phân biệt workflow và agent: workflow có code path định trước, còn agent tự quyết định process/tool usage [22]. Đề tài dùng workflow có kiểm soát với các nhánh xác định trước, không phải autonomous multi-agent.



Hình 4.3: Controlled workflow cho online query

Đồ thị đề xuất gồm: analyze_query, resolve_temporal,

expand_query, retrieve_parallel, fuse, rerank, expand_legal_context, check_evidence, targeted_retrieval, build_context, generate, verify, finalize, repair và abstain. Các cạnh có điều kiện: check_evidence phân nhánh complete sang build_context rồi generate, hoặc incomplete sang targeted_retrieval rồi quay lại check_evidence; verify phân nhánh valid sang finalize, repairable sang đường repair theo loại lỗi, còn lại sang abstain. Mọi đường quay lại đều tăng repair_attempts trong state; khi repair_attempts >= max_repair_attempts (config, khởi điểm 3) thì ABSTAIN. Không tồn tại vòng lặp không tăng counter.

4.12 Structured generation

Generator (Gemini 3.5 Flash) sinh câu trả lời theo schema cấp claim:

Listing 4.3: Structured output của generator

```

1 {
2   "answer_summary": "...",
3   "claims": [
4     {
5       "claim": "ừNgi đều êkhin xe mô tô ượtt đèn đội b ử ạpht êtin",
6       "claim_type": "MONETARY_PENALTY",
7       "provision_ids": [
8         "nd-168-2024__dieu-7__khoan-4__diem-b"
9       ],
10      "numbers": ["4.000.000", "6.000.000"]
11    }
12  ],
13  "missing_information": [],
14  "should_abstain": false
15 }

```

Schema dùng Pydantic với extra="forbid", ClaimType là enum đóng, claims bắt buộc tối thiểu một claim và mỗi provision_ids có tối thiểu một ID khi should_abstain=false. Các con số trong ví dụ mang tính minh họa cấu trúc dữ liệu, không phải khẳng định giá trị thực tế của văn bản. Model không được viết citation text; phần trích dẫn hiển thị được dựng từ metadata đã lưu, không phải chuỗi LLM gõ tự do. Khi draft có should_abstain=true, workflow không chuyển sang finalize mà route sang abstention với reason code ánh xạ và bảo toàn missing_information.

4.13 Verification sáu tầng

Verifier chạy theo thứ tự sáu tầng, deterministic-first; LLM judge độc lập chỉ được gọi ở L5 cho trường hợp ngữ nghĩa:

Bảng 4.2: Sáu tầng verification

Tầng	Bộ kiểm chứng	Nội dung
L1	Schema verifier	Output tuân thủ Pydantic schema cấp claim; không unknown field; summary ánh xạ atomic sang claim
L2	Citation ID verifier	provision_id tồn tại; thuộc context whitelist hoặc mở rộng hợp lệ; review_status = ACCEPTED; metadata citation có thẩm quyền
L3	Temporal verifier	Provision trích dẫn hợp lệ tại ngày được hỏi
L4	Numeric grounding verifier	Mức phạt, số điểm trừ, ngày, tuổi, thời hạn, số lượng khớp giá trị bằng chứng đã chuẩn hóa
L5	Claim support verifier	Deterministic trước; LLM judge độc lập chỉ cho trường hợp ngữ nghĩa, fail-closed
L6	Evidence completeness verifier	Mọi loại bằng chứng trong evidence plan được claims cuối bao phủ

Aggregate validity chỉ đúng khi cả sáu tầng đều pass, không có ``pass mềm''.
Bất biến API: Returned Invalid Citation Rate = 0. Nếu hệ thống không thể bảo đảm invariant này, request phải abstain hoặc trả lỗi.

Failure-aware repair xử lý lỗi theo loại cụ thể, không chỉ regenerate: thiếu bằng chứng chạy targeted retrieval rồi dừng lại context và regenerate; claim không được hỗ trợ regenerate từ bằng chứng hiện có hoặc targeted retrieval nếu thiếu; schema sai regenerate structured output; xung đột thời gian truy xuất phiên bản thời gian đúng. Mọi nhánh repair cùng tính vào MAX_REPAIR_ATTEMPTS; hết giới hạn thì ABSTAIN.

4.14 Abstention

Các reason code chuẩn gồm:

- OUT_OF_SCOPE, MISSING_QUERY_DATE;
- INSUFFICIENT_EVIDENCE, NO_VALID_PROVISION;
- TEMPORAL_CONFLICT, CITATION_VERIFICATION_FAILED;
- CORPUS_NOT_COVERED.

Response abstention không chứa answer pháp lý, citations rỗng, có disclaimer, có trace ID, có message rõ ràng và thông tin cần bổ sung. Hệ thống không hiển thị confidence phần trăm nếu chưa được calibration.

4.15 Langfuse observability

Langfuse phục vụ tracing, token usage, cost, latency, prompt management và experiments. Trace legal_query gồm các span analyze_query, normalize_query, rewrite_query, hyde, exact_lookup, dense_retrieval, sparse_retrieval, rrf_fusion, reranker, reference_expansion, evidence_check, generate, citation_verify, numeric_verify, claim_verify.

Langfuse không nằm trên đường tới hạn tính đúng đắn: trace được ingest bất đồng bộ, toàn bộ callback non-mutating; nếu Langfuse không khả dụng, query vẫn hoạt động bình thường. Observability giải thích hệ thống chạy như thế nào và chi phí ra sao, nhưng không thay thế logic correctness - tính đúng đắn do retrieval, evidence gate và verifier L1-L6 bảo đảm. Prompt được quản lý theo version và label production/dev; version prompt được ghi vào từng evaluation run và QueryTrace.config_snapshot.

4.16 Background ingestion và object storage

Ingestion chạy hoàn toàn qua hàng đợi: POST/api/v1/documents trả 202 Accepted kèm ingestion_job_id; không parse PDF đồng bộ trong request handler. Redis làm broker cho Dramatiq và cache; job state nằm trong PostgreSQL (ingestion_runs) là source of truth. Mỗi bước là một actor idempotent ngắn (parse, normalize, extract, resolve_refs, resolve_temporal, quality_gate, embed, index) có time limit cấu hình riêng; retry chỉ áp dụng cho lỗi transient. Worker fail được phục hồi bằng retry, dead-letter queue và script reconcile giữa job state và index state.

MinIO là object storage S3-compatible, lưu PDF nguồn, đầu ra parser, ảnh trang và artifact ingestion/review/evaluation trong các bucket riêng (source-pdfs, parser-outputs, page-images, ingestion-artifacts, review-artifacts, evaluation-artifacts). PostgreSQL lưu object key và metadata; backup MinIO dùng replication hoặc mc mirror sang nơi lưu trữ độc lập, không dùng tiering/ILM làm backup.

4.17 API và giao diện

FastAPI cung cấp chat, search, provision, documents, jobs, reviews, feedback, health, evaluations và corpus-qa endpoints [23]. Mọi response nghiệp vụ chứa trace_id; abstention dùng HTTP 200 kèm trạng thái ABSTAINED; lỗi kỹ thuật dùng 4xx/5xx.

Bảng 4.3: API chính

<i>Method</i>	<i>Path</i>	<i>Vai trò</i>
<i>POST</i>	<i>/api/v1/chat</i>	<i>Verified answer hoặc abstention</i>
<i>POST</i>	<i>/api/v1/search</i>	<i>Search provision, không gọi generator</i>
<i>POST</i>	<i>/api/v1/documents</i>	<i>Upload PDF + manifest, trả 202 + job ID</i>
<i>GET</i>	<i>/api/v1/jobs/{job_id}</i>	<i>Trạng thái job ingestion</i>
<i>GET</i> <i>POST</i>	<i>/api/v1/reviews</i>	<i>Danh sách và quyết định review</i>
<i>POST</i>	<i>/api/v1/feedback</i>	<i>Feedback người dùng cuối</i>
<i>POST</i>	<i>/api/v1/evaluations</i>	<i>Chạy evaluation run</i>
<i>GET</i>	<i>/api/v1/corpus-qa/repository</i>	<i>Báo cáo chất lượng corpus</i>
<i>GET</i>	<i>/api/v1/health/ready</i>	<i>Readiness PostgreSQL/Qdrant/Redis/MinIO</i>

Frontend Next.js có chat page, search page, citation card dựng từ metadata, passage viewer mở snippet nguồn kèm trang, date selector hiển thị ngày áp dụng, abstention panel và feedback widget. Giao diện không stream raw token vì draft có thể chưa pass verifier; SSE chỉ stream progress event như query analyzed, retrieval completed, generation started và verification completed.

4.18 Bảo mật

Admin endpoint dùng Bearer token trong P0. Upload được kiểm tra MIME, magic bytes, size, filename và SHA-256 duplicate; filename nội bộ do hệ thống sinh để chặn path traversal. PDF text được coi là data, không phải instruction, nhằm giảm prompt injection; nội dung ``ignore previous instructions'' không được làm thay đổi hành vi và claim không được hỗ trợ bị verifier từ chối. API key không được log hoặc đưa vào image. Rate limiting cấu hình theo deployment, không hardcode theo free-tier quota. Hệ thống chỉ mô tả biện pháp giảm thiểu dữ liệu cá nhân, không tuyên bố tuân thủ pháp lý tuyệt đối khi chưa có compliance review [24].

CHƯƠNG 5

CÀI ĐẶT VÀ TRIỂN KHAI

5.1 Tech stack

Bảng 5.1: Công nghệ triển khai

Thành phần	Công nghệ
Ngôn ngữ/backend	Python 3.11, FastAPI, Pydantic v2
Workflow	LangGraph 1.x controlled workflow
Parser chính	Docling 2.x
Parser phụ/fallback	MinerU 3.4.x (Parser Router)
Metadata/versioning	PostgreSQL 18, SQLAlchemy 2, Alembic
Retrieval	Qdrant v1.19 dense + sparse + RRF + payload filter
Dense embedding	Ứng viên: Gemini Embedding 2, Jina Embeddings v5 text-nano/small
Reranker	Jina Reranker v3 (stage chuẩn)
Generator	Gemini 3.5 Flash
Judge độc lập	GPT-5.4 mini snapshot [20]
Background jobs	Redis 8 + Dramatiq 2.x
Object storage	MinIO (S3-compatible)
Observability	Langfuse Cloud
Evaluation	Deterministic metrics + Ragas v0.4.x
Frontend	Next.js, TypeScript, shadcn/ui
Deployment	Docker Compose, GitHub Actions
Package manager	uv

Model ID nằm trong cấu hình, không hardcode trong domain logic. Embedding production chưa được chốt vĩnh viễn; quyết định dựa trên bảng chứng Suite B. Không dùng pgvector trong thiết kế này: vector retrieval nằm trong Qdrant, PostgreSQL giữ metadata và quan hệ pháp lý.

5.2 Cấu trúc source code

Source được chia thành API, domain, ingestion, retrieval, query, workflow, generation, verification, persistence, evaluation, feedback và observability. Domain

không import FastAPI, SQLAlchemy, Qdrant client hoặc provider SDK để giữ khả năng kiểm thử độc lập.

Listing 5.1: Cấu trúc backend rút gọn

```
1 backend/app/
2   api/
3   domain/
4   ingestion/
5     parser_router.py
6     adapters/          # docling_adapter, mineru_adapter
7     document_ir.py     # ParsedDocument, ParsedPage, DocumentElement
8     structure_extractor.py
9     context_enricher.py
10    reference_resolver.py
11    temporal_resolver.py
12    quality_gates.py
13    actors/            # parse, extract, resolve_refs, quality_gate,
14                        embed, index
15    indexer.py
16    retrieval/         # embedding, sparse, qdrant_store, hybrid,
17                        reranker
18    query/             # query_understanding, expansion, hyde,
19                        evidence_gate
20    workflow/          # graph.py, state.py, nodes/
21    generation/        # provider, gemini, prompts, schemas
22    verification/      # ll_schema ... l6_evidence
23    persistence/       # database, repositories, models
24    evaluation/        # runner, suites, deterministic_metrics,
25                        ragas_metrics
26    feedback/
27    observability/     # logging, tracing (Langfuse)
```

5.3 Cài đặt ingestion

Ingestion service nhận PDF và manifest, tính SHA-256, kiểm tra duplicate, lưu source file lên MinIO, enqueue parse_actor và trả 202 Accepted. Worker chạy pipeline parse, normalize, legal extract, reference resolve, temporal resolve, quality gates; kết quả accepted được lưu vào PostgreSQL rồi embed và upsert vào Qdrant. Qdrant upsert chạy sau PostgreSQL transaction commit; nếu upsert fail, job giữ trạng thái INDEXING và được retry bằng background/CLI reconcile, không rollback dữ liệu PostgreSQL. Actor được thiết kế idempotent: đọc state job từ PostgreSQL, bỏ qua bước đã hoàn thành.

5.4 Cài đặt retrieval

Exact lookup chuyển định danh như ``Nghị định 168, Điều 7`` thành payload filter. Dense và sparse prefetch được thực hiện song song trong Qdrant Query API; RRF fusion trả candidate; backend áp dụng temporal invariant lần

cuối để correctness không phụ thuộc một filter implementation cụ thể. Config retrieval được version hóa:

Listing 5.2: Ví dụ retrieval config

```
1 retrieval:
2   exact_lookup:
3     enabled: true
4     filter_fields: [document_number, article, clause, point]
5   dense_prefetch: 30
6   sparse_prefetch: 30
7   rrf:
8     k: 60
9     weights: {dense: 1.0, sparse: 1.0}
10  fusion_limit: 20
11  final_top_k: 8
12  temporal_filter: true
13  dedup_key: provision_id
14  reranker: jina-reranker-v3
```

Các con số là khởi điểm, chỉ được khóa sau ablation Suite C.

5.5 Cài đặt verifier

Verifier không parse citation regex. Nó nhận Pydantic object và context map. Mỗi tầng L1-L6 là một module riêng trả LayerResult gồm passed, issues, checked_claim_indexes, checked_provision_version_ids và repair_action. L4 chuẩn hóa số tiền, điểm, ngày trước khi so sánh với giá trị bằng chứng. L5 chạy deterministic trước, LLM judge độc lập (GPT-5.4 mini snapshot) chỉ cho trường hợp ngữ nghĩa với hành vi fail-closed. Returned invalid citation là invariant bằng 0; nếu hệ thống không thể bảo đảm invariant này, request phải abstain hoặc error.

5.6 Giao diện người dùng

Vị trí chèn hình sau khi hoàn thiện giao diện hoặc xuất diagram chính thức

Hình 5.1: Màn hình chat với bộ chọn ngày, loại phương tiện và citation panel

Vị trí chèn hình sau khi hoàn thiện giao diện hoặc xuất diagram chính thức

Hình 5.2: Màn hình so sánh quy định giữa hai thời điểm

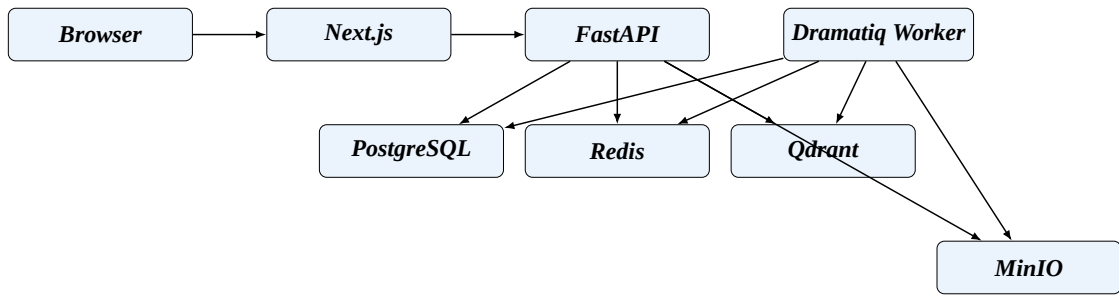
Vị trí chèn hình sau khi hoàn thiện giao diện hoặc xuất diagram chính thức

Hình 5.3: Source drawer hiển thị passage, trang và hierarchy

Citation card được dựng từ database metadata đã verify, không phải chuỗi LLM. Passage viewer mở snippet nguồn kèm trang dựa trên page number và provenance. Ngày áp dụng được hiển thị rõ trên mỗi answer. Giao diện không stream raw token vì draft có thể chưa pass verifier.

5.7 Docker Compose

Bây service chính: frontend (Next.js), backend (FastAPI), worker (Dramatiq), PostgreSQL, Qdrant, Redis và MinIO. Mọi service nằm trong một Docker network nội bộ; chỉ hai port public local là 127.0.0.1:3000 (frontend) và 127.0.0.1:8000 (backend). One-shot service migrate chạy alembic upgrade head, tạo collection Qdrant và bucket MinIO trước khi backend/worker/frontend khởi động. RAGFlow chạy trong môi trường benchmark riêng, không nằm trong compose production.



Hình 5.4: Kiến trúc triển khai local với bảy service

Các provider bên ngoài tùy chọn gồm Langfuse Cloud, Gemini API, OpenAI API (judge) và Jina API (embedding/reranker). Trong release, image bên thứ ba được pin exact tag và thay bằng digest trước code freeze.

5.8 Migration, backup và restore

Alembic quản lý schema; migration chỉ chạy bởi one-shot service migrate, application process không bao giờ tự chạy alembic upgrade head. Release bundle gồm PostgreSQL dump, Qdrant snapshot (copy sang nơi lưu trữ độc lập), MinIO mirror, gold set, evaluation results, release manifest, git tag và checksum. Vì PostgreSQL là source of truth, Qdrant có thể rebuild từ `SELECT * FROM legal_provisions WHERE review_status = 'ACCEPTED'` bằng alias switch; snapshot chỉ rút ngắn RTO, không phải nguồn chân lý. Restore được kiểm chứng bằng script `verify_release.py` so sánh số point, hash corpus và migration revision.

5.9 CI/CD và quality gate

PR pipeline chạy lint (ruff), type check (mypy), unit test với coverage core tối thiểu 80%, integration test trên PostgreSQL/Qdrant/Redis/MinIO thật (gồm alembic upgrade head từ database rỗng), frontend build và Docker build. Full LLM evaluation chỉ chạy manual ở feature freeze và release candidate để kiểm soát chi phí và nondeterminism. Bất kỳ PR nào ảnh hưởng retrieval phải chạy retrieval regression subset, temporal regression, citation invariant và gold-set integrity.

5.10 RAGFlow baseline (external)

RAGFlow không phải nền tảng chính mà chỉ là baseline so sánh bên ngoài (FR-31). Bốn variant bắt buộc: RAGFlow default, RAGFlow + Docling, RAGFlow + MinerU và pipeline VNLrag custom legal-aware, chạy trên cùng corpus và cùng bộ câu hỏi evaluation trong môi trường benchmark riêng. Chỉ số so sánh tối thiểu gồm Recall@10, citation correctness, temporal leakage và evidence completeness. RAGFlow yêu cầu tối thiểu 4 CPU, 16 GB RAM và 50 GB disk, không chạy cùng lúc với ingestion/demo/eval nặng trên cùng máy. Kết quả baseline không bao giờ được báo cáo như kết quả thực nghiệm của VNLrag.

5.11 Ràng buộc phần cứng

Máy mục tiêu dùng Intel Core i5-1035G1, 19 GB RAM và NVIDIA MX330 2 GB VRAM. Online stack chạy CPU và gọi model qua API; GPU local không dùng được cho LLM, embedding hay reranker. Docling chạy CPU trong worker với MAX_INGESTION_WORKERS=1; MinerU chỉ dùng pipeline backend CPU, không bao giờ chạy VLM/hybrid local (cần GPU tối thiểu 8 GB VRAM). Nếu RAM thực tế vượt budget, MinerU chuyển sang remote *-http-client hoặc host tách biệt; quyết định được ghi vào tài liệu vận hành.

5.12 Kế hoạch thời gian

Bảng 5.2: Mốc triển khai chính

Thời gian	Mốc	Deliverable
20/07-26/07	Chốt thiết kế và corpus	ADR, manifest, schema, Parser Router, Canonical Document IR
27/07-02/08	Parser foundation	Suite A (P1-P3), adapter Docling/MinerU, quality gates
03/08-09/08	Legal extraction và data platform	Legal Structure Extractor, corpus QA, PostgreSQL, Qdrant, MinIO, ingestion queue
10/08-16/08	Relation graph và temporal	Reference Resolver, Temporal Resolver, parent-context enrichment
17/08-23/08	Retrieval và query workflow	Suite B và C (R1-R10), query expansion, reranker, evidence gate, LangGraph skeleton
24/08-30/08	Verification, Langfuse, frontend	Verifier L1-L6, LangGraph full flow, trace, chat UI, feedback
31/08-06/09	Evaluation và stabilization	Gold set 200 câu, Suite D (G1-G7), RAGFlow baseline, feature freeze
07/09-12/09	Finalization	Code freeze, final evaluation, report, release candidate
13/09-14/09	Rehearsal/defense	Demo local và bảo vệ

Mốc kiểm soát cứng: feature freeze 06/09/2026, code freeze 10/09/2026, release candidate 12/09/2026. Không thêm tính năng mới trong hai ngày cuối; không thu hẹp phạm vi vì lịch trình.

CHƯƠNG 6

KIỂM THỬ VÀ ĐÁNH GIÁ

6.1 Chiến lược kiểm thử

Test pyramid gồm unit, integration, regression và E2E. Unit test tập trung manifest, parser, structure extractor, stable ID, temporal interval, verifier L1-L6, metric và abstention; coverage core deterministic tối thiểu 80%. Integration test bắt buộc chạy trên PostgreSQL, Qdrant, Redis và MinIO thật (không dùng SQLite để kết luận integration pass). Regression test ngăn chất lượng retrieval/citation/temporal giảm. E2E kiểm tra current, historical, comparison, abstention, multi-evidence, numeric-grounding-fail và scan PDF qua Parser Router.

6.2 Test extraction và parser

Golden fixture bao phủ Điều không có Khoản, Điều có nhiều Khoản, Khoản có nhiều Điểm, Điểm kéo dài nhiều paragraph, Chương và Mục, heading xuống dòng, OCR spacing bất thường, số thứ tự giả trong bảng, Phụ lục, nhãn d) và đ) không bị lẫn, Điểm ngăn hợp lệ, header/footer lặp, điều khoản chuyển tiếp và d/đ ambiguity. Invariant bắt buộc: source_text không rỗng, article order deterministic, không có hai provision trùng stable ID, d/đ ambiguity không tạo provision ID collision. Hierarchy extraction được chấm bằng tuple document/article/clause/point; fixture stable-ID bắt buộc phân biệt diem-d và diem-đ.

Test Parser Router kiểm tra ma trận routing: PDF searchable sang Docling; PDF scan sang Docling rồi quality gate fail sang MinerU; bảng phức tạp so sánh đầu ra hai parser. Quality gate nhóm A và nhóm B có test riêng; khi structural gate fail, dữ liệu structural cũ bị hủy và chạy lại từ parser thay thế, không trộn kết quả hai parser.

6.3 Test temporal và citation

Boundary test chạy tại effective_from, effective_to trừ một ngày và effective_to (upper bound exclusive). Temporal and Amendment Resolver được test với fixture sự kiện AMENDED, PARTIAL_AMENDED, REPEALED, SUPERSEDED và CORRECTED; mỗi event test kèm một query current và một query historical để khẳng định phiên bản đúng được chọn. Citation verifier có test cho ID ngoài context, provision không tồn tại, document chưa accepted, date invalid, claim không citation, số liệu sai (L4) và claim không được hỗ trợ (L5). Test repair workflow khẳng định mọi nhánh repair tăng cùng counter repair_attempts,

hết giới hạn kết thúc ở ABSTAIN và không có vòng lặp vô hạn.

6.4 Thiết lập evaluation

Mỗi run đọc frozen corpus, frozen gold set và variant config; result được align bằng question ID, không dùng zip; missing result làm run failed. Run và raw artifact bất biến/append-only: mỗi run ghi run_manifest_hash, model IDs, prompt versions, parser versions, token usage, estimated cost và raw results path. Deterministic metric là headline; GPT-5.4 mini snapshot chỉ chấm metric semantic phụ; citation, temporal và numeric metrics được tính bằng code. Dev set dùng để lặp phát triển, validation set dùng để chọn ngưỡng/model/prompt, final test set đóng băng và không dùng để tuning.

6.5 Thiết kế thí nghiệm các suite

6.5.1 Suite A - Parser benchmark (P1-P3)

Nội dung chờ cập nhật: Chạy P1 (Docling), P2 (MinerU) và P3 (Parser Router) trên cùng fixture với gold annotation; điền Article/Clause/Point P/R/F1, Short Point Recall, Vietnamese đ) Recall, Parent Context Completeness, Table Preservation, Header/Footer Leakage và Provenance Coverage.

6.5.2 Suite B - Embedding benchmark (E1-E3)

Nội dung chờ cập nhật: Chạy E1 (Gemini Embedding 2), E2 (Jina v5 text-nano) và E3 (Jina v5 text-small) trên development set; điền Recall@10, MRR@10, nDCG@10, latency và cost. Quyết định embedding production dựa trên kết quả này.

6.5.3 Suite C - Retrieval ablation (R1-R10)

Nội dung chờ cập nhật: Chạy các variant R1-R10 trên validation set; điền Recall@5/10/20, MRR@10, nDCG@10 cho từng variant và phân tích tăng nào đóng góp nhiều nhất. Không chọn config bằng final result.

6.5.4 Suite D - Generation và verification ablation (G1-G7)

Nội dung chờ cập nhật: Chạy G1-G7 trên validation set; điền Citation Precision/Recall/F1, Returned Invalid Citation Rate, Numeric Grounding Accuracy, Unsupported Claim Rate, Evidence Completeness và Abstention P/R/F1 cho từng variant.

6.6 Kết quả extraction

Nội dung chờ cập nhật: Điền bảng Suite A: Article/Clause/Point P/R/F1, Short Point Recall, Vietnamese đ) Recall, Parent Context Completeness, Table Preservation, Header/Footer Leakage và Provenance Coverage sau khi review bộ fixture cuối.

Bảng 6.1: Kết quả hierarchy (Suite A) - chờ final evaluation

Phương pháp	Article			Clause			Point		
	P	R	F1	P	R	F1	P	R	F1
P1 Docling	--	--	--	--	--	--	--	--	--
P2 MinerU	--	--	--	--	--	--	--	--	--
P3 Parser Router	--	--	--	--	--	--	--	--	--

Bảng 6.2: Kết quả các chỉ số bổ sung (Suite A) - chờ final evaluation

Phương pháp	Short Point	VN đ)	Parent Ctx	Table	H/F Leak	Provenance
	Recall	Recall	Completeness	Preservation	(mục tiêu thấp)	Coverage
P1 Docling	--	--	--	--	--	--
P2 MinerU	--	--	--	--	--	--
P3 Parser Router	--	--	--	--	--	--

6.7 Kết quả retrieval

Nội dung chờ cập nhật: Chạy retrieval-only trên final test set đã đóng băng.
Không chọn config bằng final result.

Bảng 6.3: Kết quả retrieval - chờ final evaluation

<i>Variant</i>	<i>R@5</i>	<i>R@10</i>	<i>MRR@10</i>	<i>nDCG@10</i>	<i>Evidence Set Recall</i>
<i>R1</i>	--	--	--	--	--
<i>R2</i>	--	--	--	--	--
<i>R3</i>	--	--	--	--	--
<i>R4</i>	--	--	--	--	--
<i>R5</i>	--	--	--	--	--
<i>R6</i>	--	--	--	--	--
<i>R7</i>	--	--	--	--	--
<i>R8</i>	--	--	--	--	--
<i>R9</i>	--	--	--	--	--
<i>R10</i>	--	--	--	--	--

6.8 Kết quả temporal và citation

Nội dung chờ cập nhật: Điền kết quả G4-G7, phân biệt Draft Invalid Citation Rate và Returned Invalid Citation Rate; báo Temporal Leakage Rate theo category.

Bảng 6.4: Kết quả temporal và citation - chờ final evaluation

<i>Variant</i>	<i>Temporal Acc.</i>	<i>Cit. P</i>	<i>Cit. R</i>	<i>Cit. F1</i>	<i>Returned Invalid</i>
<i>G4</i>	--	--	--	--	--
<i>G5</i>	--	--	--	--	--
<i>G6</i>	--	--	--	--	--
<i>G7</i>	--	--	--	--	--

6.9 Kết quả generation và abstention**Bảng 6.5: Kết quả generation - chờ final evaluation**

<i>Variant</i>	<i>Numeric Acc.</i>	<i>Unsupported Claim</i>	<i>Ev. Completeness</i>	<i>Abstention F1</i>
<i>G6</i>	--	--	--	--
<i>G7</i>	--	--	--	--

6.10 Latency, tài nguyên và chi phí

Bảng 6.6: Kết quả vận hành - chờ benchmark

Cấu hình	Retrieval P95	E2E P95	RAM đỉnh	Cost/query
R10 local defense	--	--	--	--

Các ngưỡng mục tiêu của hệ thống: retrieval P95 không quá 2 giây, end-to-end P50 không quá 12 giây, end-to-end P95 không quá 20 giây, search API P95 không quá 3 giây. Đây là mục tiêu kỹ thuật trong môi trường test, không phải kết quả đo.

6.11 Phân tích lỗi

Failure được gán một root cause trong các nhóm parser routing, parser OCR/layout, IR normalization, hierarchy extraction, provenance, dense retrieval, sparse retrieval, fusion, query analysis, query expansion, reranking, context expansion, evidence gate, temporal filter, generation, citation selection, numeric grounding, claim support, abstention, gold set và provider.

Nội dung chờ cập nhật: Sau final run, chọn 10-15 lỗi đại diện, mô tả triệu chứng, nguyên nhân, ảnh hưởng và hướng sửa. Không loại câu khó khỏi báo cáo.

6.12 Diễn giải kết quả

Khi có số liệu, phần diễn giải phải trả lời trực tiếp RQ1-RQ6. Cần báo cả absolute difference, per-category difference và failure trade-off. Ví dụ, verifier sáu tầng có thể giảm invalid citation nhưng tăng over-abstention; đây là trade-off cần báo cáo thay vì chỉ trình bày một điểm tổng hợp. Mọi báo cáo aggregate bắt buộc kèm phân rã theo category (CURRENT, HISTORICAL, COMPARISON, MULTI_PROVISION, ...) vì một điểm trung bình có thể che lỗi của nhóm câu quan trọng.

6.13 Tiêu chí release

Release candidate chỉ được chấp nhận khi migration từ database xong, restore từ release bundle, ba luồng E2E (hiện hành, lịch sử, so sánh), abstention flow, contract Returned Invalid Citation Rate = 0 và regression subset đều đạt. Báo cáo và slide phải dùng cùng evaluation artifact để tránh số liệu không đồng nhất. Trước khi nộp, chuyển \draftmodefalse và điền mọi bảng kết quả từ evaluation run đã đóng băng.

CHƯƠNG 7

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết luận

Khóa luận đề xuất một kiến trúc RAG chuyên biệt cho pháp luật giao thông đường bộ Việt Nam, tập trung vào structure-aware parsing, mô hình quan hệ tham chiếu, temporal retrieval và verifiable citations. Parser Router và Canonical Document IR cô lập tầng parsing khỏi phân tích pháp lý; Legal Structure Extractor xử lý phân cấp pháp luật Việt Nam với nhãn Điểm tiếng Việt; PostgreSQL quản lý version, quan hệ và review; Qdrant thực hiện retrieval đa tầng với exact lookup, dense, sparse và RRF; LangGraph điều phối workflow có giới hạn; generator chỉ được chọn provision_id từ context; verifier sáu tầng quyết định trả verified answer hay abstain.

Thiết kế này loại bỏ các thành phần khó kiểm soát của phương án cũ như web fallback, citation tự do, index retrieval tách rời và vòng lặp điều phối/sửa lỗi không giới hạn. Hệ thống được tổ chức để một lỗi có thể được truy ngược về parser, extraction, retrieval, temporal metadata, generation hoặc verification. Langfuse cung cấp observability nhưng không nằm trên đường tới hạn tính đúng đắn; RAGFlow chỉ đóng vai trò baseline so sánh bên ngoài.

Nội dung chờ cập nhật: Trước khi nộp, bổ sung kết luận định lượng dựa trên kết quả bốn suite A-D: phương pháp nào tốt nhất, mức cải thiện, returned invalid citation, temporal accuracy, evidence completeness, latency và chi phí.

7.2 Ưu điểm

- mô hình hóa hierarchy, quan hệ tham chiếu và temporal interval như dữ liệu nghiệp vụ trong PostgreSQL;
- Parser Router và Canonical Document IR cho phép đánh giá và thay parser mà không viết lại Legal Structure Extractor;
- nhận diện nhãn Điểm tiếng Việt d) và đ) với stable ID tái tạo được;
- provenance đến page/bounding box và truy vết về IR;
- retrieval đa tầng trong một vector database với exact lookup, dense, sparse, RRF và reranker;
- evidence planning và Evidence Completeness Gate chặn câu trả lời nửa vời;

- *citation-by-ID* và *verification* sáu tầng với *invariant Returned Invalid Citation Rate = 0*;
- *verified-or-abstain contract* với *failure-aware repair* có giới hạn;
- *evaluation* có *split*, *hash* và *raw artifacts* bất biến;
- *local-first deployment* với *Docker Compose* bày *service* và *restore plan*.

7.3 Hạn chế

- *corpus* nhỏ và chỉ tập trung giao thông đường bộ;
- *partial amendment mapping* cần nhiều *review* thủ công;
- một *reviewer* chính có thể tạo *annotation bias*;
- *model API* và *stable alias* vẫn có thể *drift* giữa các lần chạy;
- *claim support judge* chưa phải *legal expert*;
- *MinerU VLM/hybrid backend* không khả thi trên máy cá nhân, chỉ dùng *pipeline CPU*;
- hệ thống là công cụ hỗ trợ tra cứu, không thay tư vấn pháp lý.

7.4 Hướng phát triển

Hướng ngắn hạn gồm *admin review UI*, *conversation history*, *diff viewer*, *calibrated abstention* và khóa ngưỡng *reranker/evidence gate* sau khi có kết quả *Suite C-D*. Hướng trung hạn gồm *provision-level amendment engine*, *domain expert workflow*, mở rộng *baseline RAGFlow* sang các *dataset* mới và *multilingual query*. Hướng dài hạn gồm *citation entailment model* tiếng Việt, mở rộng sang các *domain pháp luật* khác, nghiên cứu *late-interaction/ColBERT-style reranking* và tái sử dụng *gold set* cho các bộ dữ liệu công khai.

7.5 Bài học phương pháp

Các nhận định như *parser* nào tốt hơn, *hybrid retrieval* tốt hơn *dense-only*, *reranker* cải thiện chất lượng hay *verifier* giảm *invalid citation* chỉ được kết luận sau thực nghiệm trên *gold set* đã *review*, không dựa trên tuyên bố nhà cung cấp. Bài học cốt lõi của thiết kế là chất lượng hệ thống pháp luật phụ thuộc đồng thời vào dữ liệu có nguồn được *review*, ranh giới pháp lý trùng ranh giới trích dẫn, *retrieval* áp dụng phiên bản hiệu lực, quan hệ tham chiếu được mở rộng có giới hạn và *contract* không cho phép *answer* chưa *verify* đi qua.

TÀI LIỆU THAM KHẢO

- [1] Quốc hội. Luật Trật tự, an toàn giao thông đường bộ số 36/2024/QH15. June 27, 2024. URL: <https://vbpl.vn/TW/Pages/vbpq-toanvan.aspx?ItemID=170620> (visited on 07/19/2026).
- [2] Chính phủ. Nghị định số 168/2024/NĐ-CP quy định xử phạt vi phạm hành chính về trật tự, an toàn giao thông trong lĩnh vực giao thông đường bộ; trừ điểm, phục hồi điểm giấy phép lái xe. Dec. 26, 2024. URL: <https://vbpl.vn/TW/Pages/vbpq-toanvan.aspx?ItemID=173920> (visited on 07/19/2026).
- [3] Chính phủ. Nghị định số 100/2019/NĐ-CP quy định xử phạt vi phạm hành chính trong lĩnh vực giao thông đường bộ và đường sắt. Dec. 30, 2019. URL: <https://vbpl.vn/TW/Pages/vbpq-toanvan.aspx?ItemID=140152> (visited on 07/19/2026).
- [4] S. Chen, H. Yin, and S. Zhou. “LegalCiteBench: Evaluating Citation Reliability in Legal Language Models”. In: *arXiv preprint arXiv:2605.10186* (2026). DOI: 10.48550/arXiv.2605.10186. URL: <https://arxiv.org/abs/2605.10186>.
- [5] Q. T. Phúc and Contributors. UDEF: Unstructured Data Extraction Framework. Đã loại bỏ khỏi kiến trúc VNLRAG v2, chỉ giữ làm tài liệu tham khảo lịch sử. 2026. URL: <https://github.com/Quach-Truong-Phuc/BOM/tree/docs/docs-overhaul> (visited on 07/19/2026).
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [7] Google. Gemini 3.5 Flash Model Documentation. 2026. URL: <https://ai.google.dev/gemini-api/docs/models/gemini-3.5-flash> (visited on 07/19/2026).
- [8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [9] Google. Gemini Embedding 2 Model Documentation. 2026. URL: <https://ai.google.dev/gemini-api/docs/models/gemini-embedding-2> (visited on 07/19/2026).
- [10] S. Robertson and H. Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389. DOI: 10.1561/1500000019.
- [11] Qdrant. Hybrid Search and BM25 Sparse Retrieval. 2026. URL: <https://qdrant.tech/documentation/search/text-search/hybrid-search/> (visited on 07/19/2026).
- [12] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, pp. 758–759. DOI: 10.1145/1571941.1572114.

- [13] *Qdrant. Hybrid and Multi-Stage Queries*. 2026. URL: <https://qdrant.tech/documentation/search/hybrid-queries/> (visited on 07/19/2026).
- [14] *Docling Project. Chunking and Hybrid Chunking Documentation*. 2026. URL: <https://docling-project.github.io/docling/concepts/chunking/> (visited on 07/19/2026).
- [15] C. Auer, M. Lysak, A. Nassar, M. Dolfi, N. Livathinos, P. Vagenas, C. B. Ramis, M. Omenetti, F. Lindlbauer, K. Dinkla, L. Mishra, Y. Kim, S. Gupta, R. T. de Lima, V. Weber, L. Morin, I. Meijer, V. Kuropiatnyk, and P. W. J. Staar. *Docling Technical Report*. Tech. rep. IBM Research, 2024. DOI: 10.48550/arXiv.2408.09869. URL: <https://arxiv.org/abs/2408.09869>.
- [16] N. Pipitone and G. H. Alami. “LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain”. In: *arXiv preprint arXiv:2408.10343* (2024). DOI: 10.48550/arXiv.2408.10343. URL: <https://arxiv.org/abs/2408.10343>.
- [17] A.-R. Butler and U. Butler. “Legal RAG Bench: An End-to-End Benchmark for Legal RAG”. In: *arXiv preprint arXiv:2603.01710* (2026). URL: <https://arxiv.org/abs/2603.01710>.
- [18] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert. “RAGAs: Automated Evaluation of Retrieval Augmented Generation”. In: *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. 2024, pp. 150–158. URL: <https://aclanthology.org/2024.eacl-demo.16/>.
- [19] *Ragas. Ragas Metrics Documentation*. 2026. URL: https://docs.ragas.io/en/latest/concepts/metrics/available_metrics/ (visited on 07/19/2026).
- [20] *OpenAI. GPT-5.4 Mini Model Documentation*. 2026. URL: <https://developers.openai.com/api/docs/models/gpt-5.4-mini> (visited on 07/19/2026).
- [21] *PostgreSQL Global Development Group. PostgreSQL Documentation*. 2026. URL: <https://www.postgresql.org/docs/> (visited on 07/19/2026).
- [22] *LangChain. LangGraph Workflows and Agents*. 2026. URL: <https://langchain-ai.github.io/langgraph/agents/tools/> (visited on 07/19/2026).
- [23] S. Ramírez and Contributors. *FastAPI Documentation*. 2026. URL: <https://fastapi.tiangolo.com/> (visited on 07/19/2026).
- [24] *Chính phủ. Nghị định số 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân*. Apr. 17, 2023. URL: <https://vbpl.vn/TW/Pages/vbpq-toanvan.aspx?ItemID=161106> (visited on 07/19/2026).
- [25] *Khoa Kỹ thuật - Công nghệ, Trường Đại học Tây Đô. Quy định cách viết và trình bày niên luận, tiểu luận, khóa luận*. Trường Đại học Tây Đô. Cần Thơ, 2024.

CHƯƠNG A

HƯỚNG DẪN CHẠY DEMO

A.1 Khởi động hạ tầng

Theo quy định trình bày khóa luận của Khoa Kỹ thuật - Công nghệ, Trường Đại học Tây Đô [25], mọi lệnh docker compose chạy từ repo root và kèm --project-directory ..

Listing A.1: Khởi động defense mode

```
1 cp deploy/env/release.env.example .env
2 # Dien API key va secret
3
4 docker compose \
5   --project-directory . -f deploy/compose/compose.release.yml \
6   up -d
7
8 docker compose --project-directory . \
9   -f deploy/compose/compose.release.yml wait backend frontend
10
11 curl --fail http://localhost:8000/api/v1/health/ready
```

A.2 Kịch bản demo

1. Hỏi quy định hiện hành.
2. Mở citation và source passage kèm trang.
3. Hỏi cùng hành vi tại thời điểm lịch sử.
4. So sánh trước và sau một mốc hiệu lực.
5. Hỏi ngoài corpus để minh họa abstention.
6. Hỏi câu đa bằng chứng (mức phạt + điểm trừ) để minh họa evidence completeness.
7. Mở bảng kết quả evaluation bốn suite A-D.

CHƯƠNG B

SCHEMA GOLD SET

Listing B.1: Gold record schema rút gọn

```
1 {
2   "id": "Q001",
3   "split": "FINAL_TEST",
4   "question": "ăNm 2023 xe máy ượvt đèn đội b ưở lý êth nào?",
5   "category": "HISTORICAL",
6   "query_date": "2023-07-01",
7   "vehicle_type": "MOTORCYCLE",
8   "expected_provision_ids": ["..."],
9   "acceptable_provision_ids": ["..."],
10  "required_evidence": ["monetary_penalty"],
11  "must_include_facts": ["..."],
12  "must_not_include_facts": ["..."],
13  "temporal_metadata": {
14    "query_date": "2023-07-01",
15    "date_source": "canonical_date"
16  },
17  "review_status": "REVIEWED",
18  "reviewed_by": "reviewer-01",
19  "gold_version": "gold-v1",
20  "hash": "sha256:..."
21 }
```

Mọi định danh pháp lý trong ví dụ là placeholder tổng hợp; ID thật được xác định theo quy trình tạo gold và validate trong PostgreSQL. Câu lịch sử năm 2023 phải trở tới provision của văn bản hợp lệ tại mốc 2023, không trở tới văn bản ban hành sau mốc hỏi.

CHƯƠNG C

CHECKLIST TRƯỚC KHI NỘP

- ☐ *Điền tên giảng viên hướng dẫn và mã số sinh viên.*
- ☐ *Chèn logo chính thức và screenshot giao diện.*
- ☐ *Chuyển \draftmodefalse sau khi có final results.*
- ☐ *Điền các bảng kết quả Chương 6 từ evaluation artifact.*
- ☐ *Cập nhật Tóm tắt, Abstract và Kết luận bằng số liệu thật.*
- ☐ *Kiểm tra mọi citation trong bibliography.*
- ☐ *Kiểm tra danh mục hình, bảng và mục lục.*
- ☐ *Build bằng XeLaTeX + Biber từ clean workspace.*
- ☐ *Render PDF và kiểm tra clipping, font, table overflow.*
- ☐ *Đồng bộ số liệu giữa báo cáo và slide.*
- ☐ *Tạo release bundle và video backup.*